

Univerzita Jana Evangelisty Purkyně
v Ústí nad Labem
Přírodovědecká fakulta



Porovnání databázových systémů pro
zpracování časových řad

BAKALÁŘSKÁ PRÁCE

Vypracoval: Martin Formánek

Vedoucí práce: Ing. Roman Vaibar, Ph.D., MBA

Studijní program: Aplikovaná informatika

ÚSTÍ NAD LABEM 2026

Namísto žlutých stránek vložte digitálně podepsané zadání kvalifikační práce poskytnuté vedoucím katedry.

Zadání musí zaujímat právě dvě strany.

Zadání je nutno vložit jako PDF pomocí některého nástroje, který umožňuje editaci dokumentů (se zachováním elektronického podpisu).

V Linuxu lze například použít příkaz `pdftk`.

Prohlášení

Prohlašuji, že jsem tuto bakalářskou práci vypracoval samostatně a použil jen pramenů, které cituji a uvádím v přiloženém seznamu literatury.

Byl jsem seznámen s tím, že se na moji práci vztahují práva a povinnosti vyplývající ze zákona č. 121/2000 Sb., ve znění zákona č. 81/2005 Sb., autorský zákon, zejména se skutečností, že Univerzita Jana Evangelisty Purkyně v Ústí nad Labem má právo na uzavření licenční smlouvy o užití této práce jako školního díla podle § 60 odst. 1 autorského zákona, a s tím, že pokud dojde k užití této práce mnou nebo bude poskytnuta licence o užití jinému subjektu, je Univerzita Jana Evangelisty Purkyně v Ústí nad Labem oprávněna ode mne požadovat přiměřený příspěvek na úhradu nákladu, které na vytvoření díla vynaložila, a to podle okolností až do jejich skutečné výše.

V Ústí nad Labem dne 9. července 2026

Podpis:

Děkuji vedoucímu práce Ing. Mgr. Romanovi Vaibarovi, Ph.D., MBA
za neocenitelné rady a pomoc při tvorbě bakalářské práce.

Dále děkuji své rodině a nejbližším přátelům
za podporu během studia a psaní závěrečné práce.

Abstrakt:

Abstract práce uvádí základní téma práce a především její výstupy. Rozsah abstraktu by měl být alespoň osmdesát slov (abstrakty a klíčová slova se však musí vejít na stránku).

Klíčová slova: tři až pět klíčových slov resp. termínů, která usnadní případné vyhledávání závěrečné práce, v pořadí od obecnějších ke konkrétnějším

COMPARISON OF DATABASE SYSTEMS FOR TIME SERIES PROCESSING

Abstract:

The abstract states the main topic of the thesis and mainly its outcomes. The length of the abstract should be at least eighty words (however, abstracts and keywords must fit on a page)

Keywords: three to five key words or terms to facilitate a possible search of the thesis, in order from more general to more specific

Obsah

Seznam obrázků	13
Seznam ukázek kódu	16
Seznam tabulek	17
1. Úvod	19
2. Teoretická část	21
2.1. Charakteristika časových řad	21
2.2. Ukládání časových řad	22
2.3. Databázové systémy	24
2.4. Úvod do problematiky testování a implementace	30
3. Praktická část	41
3.1. Popis dat	41
3.2. Předběžná srovnávací analýza systémů	41
3.3. Návrh testovacího scénáře	48
3.4. Konfigurace testovacího prostředí a databázových systémů	49
3.5. Pilotní nasazení systémů	56
3.6. Implementace testů	57
4. Výsledky výzkumu a diskuse	63
5. Závěr	65
Seznam použitých zdrojů	69
A. Architektura Ansible	71
B. Architektura Prometheus	81
C. Externí přílohy	85

Seznam obrázků

2.1.	Ukázka časové řady (zdroj: vlastní zpracování)	22
2.2.	Ukázka struktury SQL tabulky (zdroj: Vlastní zpracování)	25
2.3.	Ukázka struktury databáze párů klíč-hodnota (zdroj: vlastní zpracování)	26
2.4.	Ukázka struktury dokumentové databáze (zdroj: vlastní zpracování)	27
2.5.	Ukázka struktury sloupcové databáze (zdroj: vlastní zpracování)	27
2.6.	Ukázka struktury grafové databáze (zdroj: vlastní zpracování)	28
2.7.	Ukázka organizace dat v B-Tree struktuře (převzato z [3])	29
2.8.	Ukázka organizace dat v LSM stromu (převzato z [12])	29
2.9.	Virtuální stroj a kontejner (zdroj: vlastní zpracování)	30
2.10.	Porovnání Typu 1 a Typu 2 hypervizorů (zdroj: vlastní zpracování)	31
3.1.	Ukázka komunikace mezi uzly (zdroj: vlastní zpracování)	52
B.1.	Ukázka metriky typu Counter (zdroj: vlastní zpracování)	82
B.2.	Ukázka metriky typu Gauge (zdroj: vlastní zpracování)	82
B.3.	Ukázka metriky typu Histogram (zdroj: vlastní zpracování)	83

Seznam ukázek kódu

3.1.	Instalace Ansible	50
3.2.	Instalace kolekce community.general	50
3.3.	inventory/local	51
3.4.	Výchozí proměnné role clickhousedb	53
3.5.	roles/clickhousedb/tasks/install.yml (zkráceno)	54
3.6.	roles/clickhousedb/handlers/main.yml	55
3.7.	Výřez z roles/clickhousedb/tasks/setup_database.yml	55
3.8.	Příklad spuštění live insert testu	57
3.9.	Příklad spuštění live insert testu	57
3.10.	Výřez z tests/test.yml	58
3.11.	Výřez z tests/clickhousedb/helper/get_query_timing.yml	59
3.12.	Výřez z tests/clickhousedb/insert.yml	59
3.13.	Výřez z tests/clickhousedb/live_insert.yml	60
A.1.	Ukázka YAML	71
A.2.	Ukázka INI	71
A.3.	Ukázka Jinja2	72
A.4.	Ansible - ukázka úlohy (Task)	72
A.5.	Ansible - ukázka playbooku	73
A.6.	Ansible - ukázka INI inventáře	74
A.7.	Ansible - ukázka YAML inventáře	74
A.8.	Struktura adresářů pro Ansible role	75
A.9.	Použití proměnné	76
A.10.	Shromážděná fakta	77
A.11.	Proměnné přidané argumentem	77
A.12.	Proměnné při zahrnutí role	77
A.13.	Registrované proměnné	78
A.14.	Playbook proměnné	78
A.15.	Proměnné hostů	78
A.16.	Proměnné skupin	78
A.17.	Proměnné v inventáři	78
B.1.	Prometheus - ukázka konfigurace	81

B.2. PromQL - ukázka dotazů	83
---------------------------------------	----

Seznam tabulek

2.1.	Časová řada načítání webové stránky (zdroj: vlastní zpracování)	24
2.2.	Zjednodušený záznam načítání webové stránky (zdroj: vlastní zpracování)	24
2.3.	Srovnání agent-based a agent-less přístupu (zdroj: vlastní zpracování)	34
3.1.	Srovnání databázových systémů	42
3.2.	Porovnání hardwarových specifikací testovacích strojů	58

1. Úvod

Časové řady představují jeden z nejběžnějších typů dat, se kterými se v praxi setkáváme. Ať se jedná o senzorová měření, metriky výkonu softwarových systémů, finanční ukazatele nebo data o pohybu vozidel, ve všech případech jde o posloupnosti hodnot uspořádané v čase, jejichž efektivní ukládání a analýza má zásadní vliv na schopnost organizace rozhodovat se na základě reálných dat. S rostoucím množstvím připojených zařízení, senzorů a monitorovaných systémů roste i objem generovaných časových dat, což klade stále vyšší nároky na databázové systémy, které je mají ukládat a zpracovávat.

V minulosti se pro ukládání časových řad nejčastěji využívaly tradiční relační databáze, které ovšem nejsou na specifický charakter tohoto typu dat primárně optimalizované. V posledních letech proto vznikla celá řada specializovaných databázových systémů, takzvaných time-series databází, které se snaží lépe reflektovat vlastnosti časových řad, jako je vysoká frekvence zápisu, potřeba efektivní agregace dat v čase nebo nutnost dlouhodobé archivace velkých objemů dat. Zároveň se v praxi stále využívají i sloupcové databáze a relační databáze rozšířené o podporu časových řad, takže výběr vhodného systému není jednoduchý a závisí na konkrétním případě použití.

Cílem této bakalářské práce je porovnat vybrané databázové systémy z hlediska jejich vhodnosti pro zpracování a ukládání časových řad, a to jak z teoretického hlediska architektury a vnitřních datových struktur, tak i z hlediska praktického výkonu při konkrétních operacích, jako je zápis, čtení a agregace dat. Pro účely srovnání bude vytvořeno automatizované testovací prostředí, ve kterém budou jednotlivé systémy nasazeny za srovnatelných podmínek a podrobeny sadě definovaných testů.

Práce je rozdělena do dvou hlavních částí. Teoretická část se nejprve věnuje charakteristice časových řad a způsobům jejich ukládání, dále popisuje rozdíly mezi jednotlivými typy databázových systémů a jejich vnitřními datovými strukturami. Následně je čtenář seznámen s nástroji a technologiemi použitými pro automatizaci a monitorování testovacího prostředí, konkrétně s nástrojem Ansible pro automatizaci konfigurace a s nástroji Prometheus a Grafana pro sběr a vizualizaci metrik. Praktická část se pak zaměřuje na popis použitého datasetu, návrh a implementaci testovacího scénáře, samotnou konfiguraci testovaného prostředí a vyhodnocení dosažených výsledků. Výstupem práce by mělo být nejen porovnání konkrétních databázových systémů na konkrétním datasetu, ale i obecnější doporučení, na základě jakých kritérií by měl být databázový systém pro zpracování časových řad vybírán v závislosti na požadavcích konkrétního projektu.

2. Teoretická část

2.1. Charakteristika časových řad

Časové řady představují posloupnosti datových bodů uspořádaných podle času. Jednotlivá pozorování mohou být zaznamenávána v pravidelných či nepravidelných časových intervalech nebo mohou být považována za spojitě definovaná v čase. V případě, že jsou hodnoty měřeny v konkrétních a oddělených časových okamžicích, hovoříme o diskrétní časové řadě. Naopak, pokud lze sledovanou veličinu považovat za definovanou v každém okamžiku času, jedná se o kontinuální časovou řadu. [1]

Datové body časových řad mohou reprezentovat širokou škálu jevů, například teplotní záznamy, údaje z finančních trhů, senzorová měření, dopravní intenzitu nebo spotřebu energie. Hodnoty nemusí být nutně číselné, ale mohou být také kategorické, například ve formě stavů zařízení nebo typů událostí. Pozorování navíc nemusí odpovídat jedinému časovému okamžiku, ale mohou představovat agregované¹ hodnoty za určité časové období, například denní průměrnou teplotu nebo hodinový výkon výrobní linky. Délka těchto intervalů závisí na charakteru sledovaného jevu a požadavcích následné analýzy. [1]

Z matematického hlediska lze časovou řadu formálně definovat jako množinu pozorování

$$X = \{X_t : t \in T\}, \quad (2.1)$$

kde X_t představuje hodnotu sledované veličiny v čase t a T je množina časových okamžiků. [1]

Analýza

V podstatě všechny časové řady jsou zaznamenávány za účelem následné analýzy. Tato analýza může mít různé cíle, které lze shrnout do následujících kategorií:

- **Zkoumání vlivů na sledovaný jev**

Časové řady se často používají k odhalování vztahů mezi různými faktory a sledovaným jevem. Například v ekonomii mohou být analyzovány dopady změn úrokových sazeb na inflaci nebo v energetice vliv teploty na spotřebu elektrické energie. Taková analýza umožňuje pochopit, jak jednotlivé vlivy ovlivňují chování systému v čase.

¹Agregované - seskupené

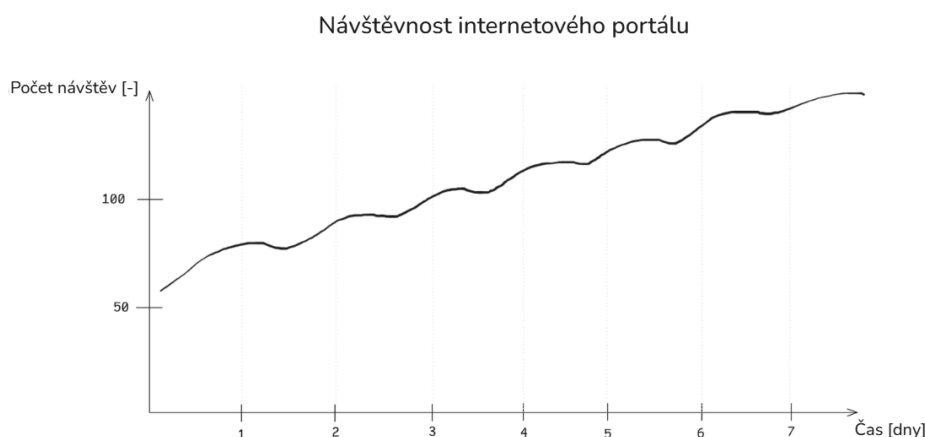
- **Zkoumání samotného jevu**

Někdy je cílem zkoumat pouze samotný jev. Například meteorologické časové řady teplot mohou odhalit dlouhodobé oteplování, sezónní výkyvy nebo extrémní jevy, jako jsou vlny horka.

- **Predikce budoucího vývoje**

Historická data umožňují předpovídat budoucí hodnoty sledovaného jevu. Taková predikce se využívá například při plánování kapacit výroby, řízení zásob nebo finančních investic.

Analýza časových řad zahrnuje různé techniky a metody, které umožňují identifikovat trendy, sezónnost, závislosti v čase a další charakteristiky dat. [1]



Obrázek 2.1.: Ukázka časové řady (zdroj: vlastní zpracování)

Na obrázku 2.1 je znázorněna ukázka časové řady, kde na vodorovné ose jsou zobrazeny časové okamžiky v rámci dní a na svislé ose hodnoty sledovaného jevu - Současných online uživatelů na webovém portálu.

Lze na ní pozorovat denní cyklus, neboli sezónnost, kdy počet uživatelů vrcholí v odpoledních hodinách a klesá během noci, a zároveň linární rostoucí trend, stránku každý den navštíví více uživatelů.

2.2. Ukládání časových řad

Mnohdy hodnota časových řad leží spíše v retrospektivní analýze než v analyzování každé nové hodnoty v reálném čase. Z tohoto důvodu je často potřeba tyto časové řady efektivně ukládat pro pozdější analýzu. Existuje mnoho různých přístupů k ukládání časových řad, od tradičních relačních databází přes specializované databázové systémy navržené speciálně pro tento účel až po obyčejné textové soubory. [2]

Při výběru správného formátu a databázového systému pro ukládání časových řad je důležité zvážit několik faktorů, jako jsou:

- **Objem dat**

Časové řady mohou a mnohdy i generují daleko větší objemy dat než jiné typy dat. Tento objem je důležité předem odhadnout a zvolit systém, který je schopný efektivně tento růst zvládnout. [3]

- **Neustálý příjem nebo jednotlivé události**

Při vybírání ideálního systému, například pro ukládání návštěvnosti webových stránek, je potřeba zvážit charakter dat. V případě neustálého toku dat je obvykle nejzajímavější pracovat s nejnovějšími záznamy, protože ty starší mohou být méně relevantní pro analýzu. Naopak pokud data představují jednotlivé události, například každoroční dopravní provoz během svátků, může být důležité mít přístup i k historickým datům. V takovém případě je pravděpodobnější, že systém bude potřebovat podporu pro náhodný přístup k různým časovým intervalům. [3]

- **Trvání projektu**

Pokud je projekt krátkodobý, může být dostačující použít jednodušší a rychle nasaditelná řešení, jako jsou textové soubory nebo jednoduché databáze. Pokud známe délku projektu, daleko snáze se odhaduje objem dat, který bude potřeba uložit. [2]

- **Zpracování a životní cyklus dat**

Je vhodné předem stanovit, jak budou data zpracovávána a jak dlouho mají být uchovávána. Ne všechny záznamy je nutné uchovávat trvale, některé lze po určité době agregovat, downsamplovat² nebo smazat. Definování jasné strategie mazání či archivace dat pomáhá předejít nekontrolovatelnému růstu databáze a zároveň usnadňuje správu a optimalizaci výkonu systému.

Tyto faktory také mohou pomoci určit, jestli je nutné ukládat neupravená data nebo předem zpracovaná.

Životnost dat

Při rozhodování o tom, jaké úložiště dat je vhodné, je klíčové porozumět životnímu cyklu dat. Čím realističtější budeme ohledně skutečného využití dat, tím méně jich je nutné ukládat a tím méně času strávíme hledáním ideálního úložného systému. Mnoho organizací zaznamenává příliš mnoho událostí a bojí se o ztrátu dat, ale mít obrovské množství špatně zpracovatelných dat je mnohem méně užitečné než mít agregovaná data uložená v relevantních časových intervalech.

U krátkodobých dat, například výkonnostních metrik, které se sledují jen kvůli ověření, že je vše v pořádku, možná vůbec nebude potřeba ukládat data v původní podobě, nebo alespoň ne na dlouho. To je zvláště důležité u dat založených na událostech, kde jednotlivé události samy o sobě nejsou významné a hlavní hodnotu mají až souhrnné statistiky. [2]

²Downsampling - proces převzorkování dat na menší počet datových bodů. [2]

Předpokládejme, že sledujeme dobu načítání webové stránky v sekundách. V tabulce 2.1 je uvedena ukázka časové řady načítání webové stránky v různých časových okamžicích.

Časové razítko	Čas načtení stránky
Duben 5, 2024 10:22:24	23s
Duben 5, 2024 10:22:28	15s
Duben 5, 2024 10:22:41	14s
Duben 5, 2024 10:23:02	11s
Duben 5, 2024 10:23:15	19s
Duben 5, 2024 10:23:33	12s

Tabulka 2.1.: Časová řada načítání webové stránky (zdroj: vlastní zpracování)

V takovém případě nás nezajímá každý jednotlivý záznam, ale spíše souhrnné statistiky, jako je průměrný čas načítání za minutu, hodinu nebo den. Proto můžeme zvolit strategii, kde budeme uchovávat pouze agregovaná data, například průměrný čas načítání za každou minutu, a původní záznamy po určité době smazat. Tímto způsobem můžeme výrazně snížit objem uložených dat a zároveň zachovat relevantní informace pro analýzu výkonu webové stránky. [2]

Výsledný záznam by tedy mohl vypadat například takto v tabulce 2.2.

Časové rozmezí	Prům. čas načtení	Počet přístupů	Min. čas	Max. čas
Duben 5, 2024 10:00:00	15.6s	6	11s	19s
Duben 5, 2024 11:00:00	17.2s	4	9s	25s

Tabulka 2.2.: Zjednodušený záznam načítání webové stránky (zdroj: vlastní zpracování)

2.3. Databázové systémy

Databázové systémy jsou ten nejběžnější způsob jak ukládat časové řady. Většinou jsou připravené tzv. "out of the box", neboli ihned po pořízení, poskytují spoustu vestavěných funkcí pro jednoduché zpracování dat, jako je například agregace a filtrování. Navíc jsou optimalizované pro rychlý zápis a čtení dat, a dokáží škálovat s rostoucím objemem dat. [2]

OLTP a OLAP databáze

Databázové systémy lze rozdělit podle typu pracovního zatížení na systémy OLTP (Online Transaction Processing) a OLAP (Online Analytical Processing). Oba přístupy jsou navrženy pro odlišné způsoby práce s daty a liší se zejména charakterem dotazů, způsobem ukládání dat a optimalizací výkonu. [4]

- **OLTP**

OLTP databáze jsou určeny pro každodenní provoz aplikací, kde probíhá velké množství krátkých transakcí, například vkládání, aktualizace nebo mazání jednotlivých záznamů. Typickými příklady jsou bankovní systémy, rezervační systémy nebo e-shopy. Tyto databáze jsou optimalizovány především pro rychlé zápisy a čtení jednotlivých řádků při zachování konzistence dat pomocí ACID³ transakcí. [4]

- **OLAP**

OLAP databáze jsou určeny pro analytické zpracování velkého objemu historických dat. Místo jednotlivých záznamů pracují s rozsáhlými datovými sadami a jsou optimalizovány pro agregace, filtrování a statistické analýzy. Typickými příklady využití jsou business intelligence, reporting nebo analýza časových řad. Často využívají sloupcové ukládání dat, které umožňuje efektivně zpracovávat dotazy nad miliony až miliardami záznamů. [4]

Databáze určené pro ukládání časových řad často kombinují vlastnosti obou přístupů. Musí zvládat vysokou rychlost zápisu nových měření podobně jako OLTP databáze, současně však umožňují rychlé analytické dotazy nad historickými daty, které jsou charakteristické pro OLAP systémy. [4]

SQL a NoSQL databáze

Dále lze databázové systémy rozdělit z hlediska datového modelu na dvě hlavní skupiny, relační (SQL) a nerelační (NoSQL) databáze. Relační databáze ukládají data do tabulek propojených vzájemnými vztahy a využívají standardizovaný jazyk SQL (Structured Query Language). Naproti tomu NoSQL databáze nejsou založeny na relačním modelu a nabízejí různé způsoby organizace dat podle požadavků konkrétní aplikace.

Hlavní rozdíly mezi oběma přístupy spočívají zejména ve způsobu organizace dat a flexibilitě datového modelu. [6]

- **SQL**

Relační databáze používají pevně definované schéma, takže každá tabulka má předem určené sloupce a jejich datové typy. Tento přístup zajišťuje vysokou konzistenci dat a je vhodný zejména v případech, kdy je struktura dat předem známá a v průběhu času se výrazně nemění. [7]

id	item	qty
509	stul	8
510	zidle	16

Obrázek 2.2.: Ukázka struktury SQL tabulky (zdroj: Vlastní zpracování)

³ ACID označuje čtyři vlastnosti databázových transakcí – atomičnost, konzistenci, izolaci a trvalost – které zajišťují správné a spolehlivé zpracování dat. [5]

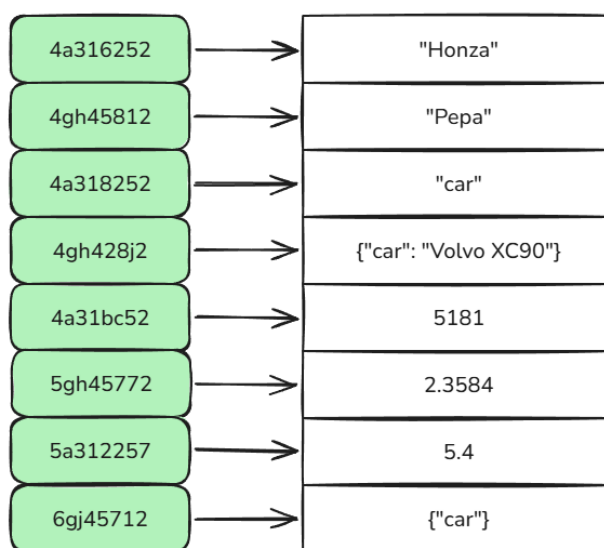
- **NoSQL**

NoSQL databáze nejsou omezeny pevným schématem, což umožňuje jednodušší přizpůsobení měnící se struktuře dat. Podle způsobu organizace dat je lze rozdělit do několika kategorií, z nichž nejčastější jsou:

- **Databáze párů klíč-hodnota**

Ukládají data jako páry klíč-hodnota, což umožňuje rychlý přístup k záznamům na základě klíče. Jsou vhodné zejména tam, kde je známý identifikátor záznamu, zatímco samotná hodnota může mít různou strukturu. Takové databáze zpravidla nedokáží například přes hodnoty filtrovat, ale zato je dokáží najít v $O(1)$ čase.

Typickým představitelem je Redis, který je často využíván pro caching⁴ a real-time aplikace jakožto dodatečná vrstva mezi uživatelem a koncovou databází. [8]



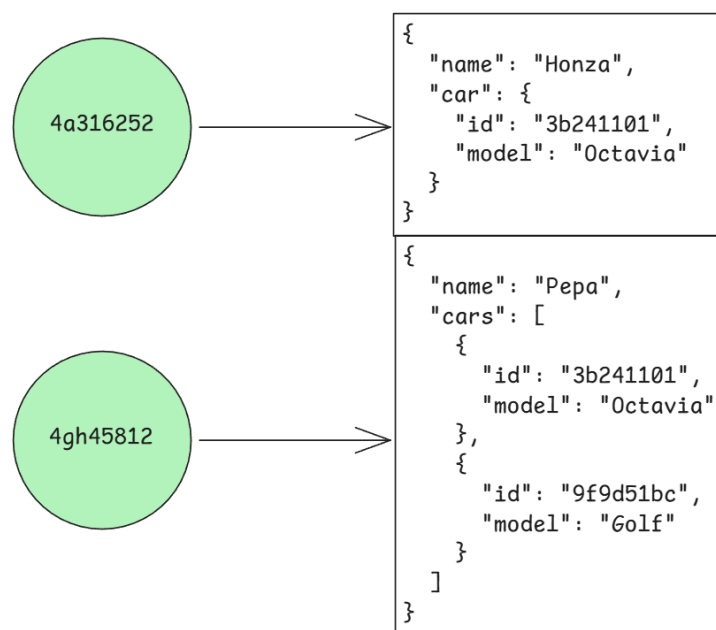
Obrázek 2.3.: Ukázka struktury databáze párů klíč-hodnota (zdroj: vlastní zpracování)

- **Dokumentová databáze**

Ukládají data ve formě dokumentů, často ve formátu JSON nebo BSON. Jednotlivé dokumenty mohou mít odlišnou strukturu a bývají sdružovány do kolekcí. Díky podpoře vnořených objektů jsou vhodné pro aplikace, u nichž se datový model často mění. Velmi dobře dokáží zpracovat právě například strukturu JSONu, takže se nimi dá například velmi snadno filtrovat.

Typickým představitelem je MongoDB. Často se používá pro webové aplikace, analytické systémy nebo logování, kde se struktura dat může často měnit. [9]

⁴Caching - technika ukládání často používaných dat do rychlé paměti



Obrázek 2.4.: Ukázka struktury dokumentové databáze (zdroj: vlastní zpracování)

– Sloupcové databáze

Ukládají data po sloupcích namísto řádků, což je výhodné zejména při analytických dotazech, pracujících pouze s vybranými atributy. Omezením čtení pouze potřebných sloupců se snižuje objem načítaných dat a zvyšuje výkon analytických operací.

Typickým představitelem je ClickHouse, který je často využíván pro analytické úlohy v reálném čase, jako je sledování metrik a logování. [10]

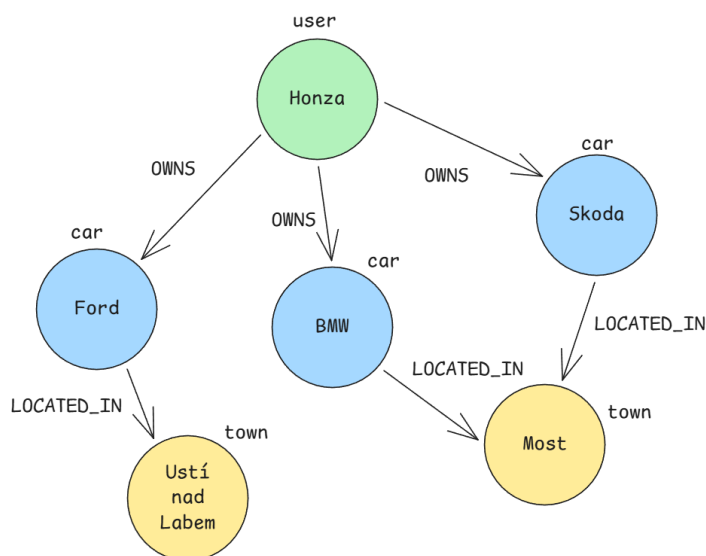
date	product	revenue
2025-11-1	Notebook	25477
2025-11-1	Myš	699
2025-11-2	Notebook	25477
2025-11-3	Klávesnice	1199
2025-11-4	Notebook	25477

Obrázek 2.5.: Ukázka struktury sloupcové databáze (zdroj: vlastní zpracování)

– Grafová databáze

Ukládají data ve formě uzlů a hran reprezentujících vzájemné vztahy mezi entitami. Jsou vhodné především pro úlohy, kde jsou vztahy mezi objekty důležitější než samotné atributy objektů.

Typickým představitelem je Neo4j, který se často používá pro sociální sítě, doporučovací systémy a analýzu podvodů. [11]



Obrázek 2.6.: Ukázka struktury grafové databáze (zdroj: vlastní zpracování)

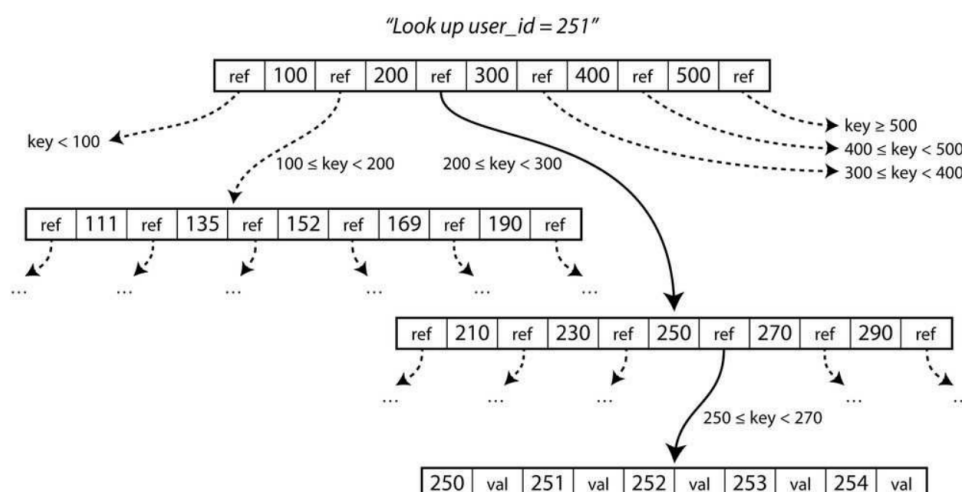
Datové struktury

Databázové systémy se liší nejen způsobem organizace dat, ale také vnitřními datovými strukturami, které používají pro ukládání a vyhledávání záznamů. Mezi ty nejběžnější patří B-Tree a LSM strom. Tyto struktury výrazně ovlivňují výkon databázového systému a určují, jak následně co nejefektivněji daný databázový systém využít.

B-Tree

B-Tree je datová struktura, která uchovává data ve formě klíč-hodnota, která jsou seřazené podle klíče.

B-Tree rozdělí úložiště dat do několika fixních bloků (často odpovídajících velikosti diskového sektoru). Každý takový blok může být lehce identifikovaný pomocí adresy a jeden blok může obsahovat více adres odkazujících na jiné bloky. [3] Jeden blok je označený za kořenový blok, který je vždy výchozím bodem pro hledání dat.



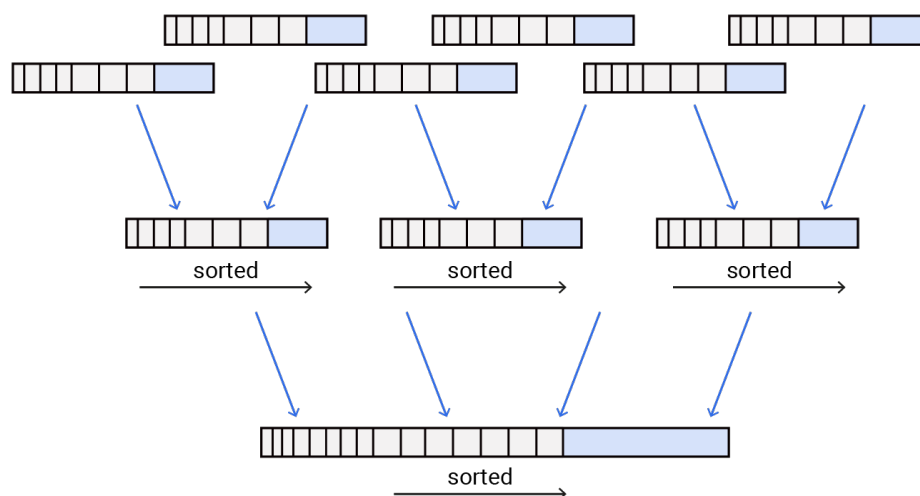
Obrázek 2.7.: Ukázka organizace dat v B-Tree struktuře (převzato z [3])

V ukázce na obrázku 2.7 hledáme záznam s klíčem 251, takže víme, že musíme sledovat odkaz do seznamu 200-299. Takový seznam se dále dělí do bloků seřazené od x-209, 210-229, 230-249, 250-269, 270-289 a 290-x. Eventuelně se dostaneme až do správného bloku který hledáme. [3]

LSM Strom

Log-Structured Merge-Tree je datová struktura optimalizovaná pro vysokou propustnost zápisů. Každý zápis se nejprve uloží do rychlé paměti (memtable) a periodicky se přepíše do trvalého úložiště (disku) ve formě seřazených souborů (SSTables). Tyto SSTables jsou přirozeně seřazené podle klíče a jsou už dále neměnné. [3]

Při čtení se pak postupuje nejdříve přečtením memtables a následně od nejnovějších SSTables k nejdřívějším, dokud se nenajde požadovaný záznam nebo se nedojde na konec. [3]

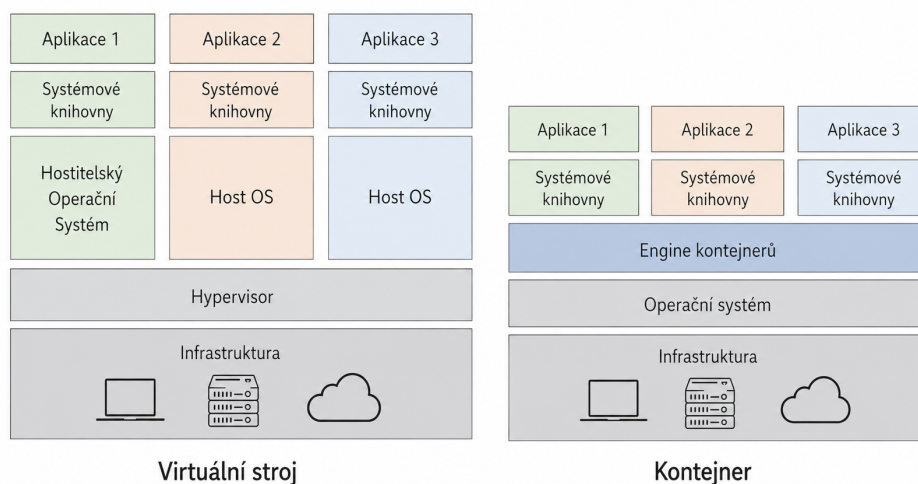


Obrázek 2.8.: Ukázka organizace dat v LSM stromu (převzato z [12])

Starší SSTables jsou pravidelně slučovány do větších souborů, aby se snížil počet souborů, které je třeba prohledávat při čtení, a aby se odstranily záznamy, které byly aktualizovány nebo smazány. Díky tomu že jsou přirozeně seřazené, je tento proces slučování efektivní. [3]

2.4. Úvod do problematiky testování a implementace

Při porovnávání databázových systémů je důležité zajistit, aby byly všechny testy prováděny ve srovnatelných podmínkách. Rozdíly v konfiguraci serverů, nainstalovaném softwaru nebo způsobu nasazení mohou výrazně ovlivnit dosažené výsledky a vést ke zkresleným závěrům. Proto je vhodné využívat izolovaná a snadno reprodukovatelná prostředí, ve kterých lze jednotlivé systémy nasazovat opakovaně a konzistentně.



Obrázek 2.9.: Virtuální stroj a kontejner (zdroj: vlastní zpracování)

Taková prostředí dělíme na dva hlavní typy:

Kontejner

Kontejner představuje standardizovanou softwarovou jednotku, která zapouzdřuje kód aplikace společně se všemi jejími závislostmi a knihovnami. Na rozdíl od virtuálních strojů kontejnery neobsahují vlastní operační systém a nevyužívají hypervizor. Místo toho sdílejí jádro hostitelského systému, což zajišťuje izolaci procesů při zachování minimální režie. Díky tomu jsou kontejnery výrazně menší (často v řádech megabajtů) a jejich spuštění je rychlejší. [13]

Klíčovou vlastností kontejnerů je jejich přenositelnost. Aplikaci zabalenou v kontejneru lze snadno přesouvat mezi různými prostředími, ať už od lokálního vývoje až po produkční nasazení v cloudu, s jistotou, že bude fungovat konzistentně bez ohledu na podkladovou infrastrukturu. O masivní rozšíření a standardizaci této technologie se v posledních letech zasloužila především platforma Docker.

Virtuální stroj

Virtuální stroj je softwarová emulace fyzického počítače, která spouští vlastní kompletní operační systém v izolovaném prostředí. Provoz více virtuálních strojů na jednom fyzickém serveru umožňuje vrstva zvaná hypervizor, která se stará o přidělování hardwarových prostředků (CPU, paměť, úložiště) jednotlivým instancím. [13] Hypervizory lze rozdělit do dvou hlavních kategorií:

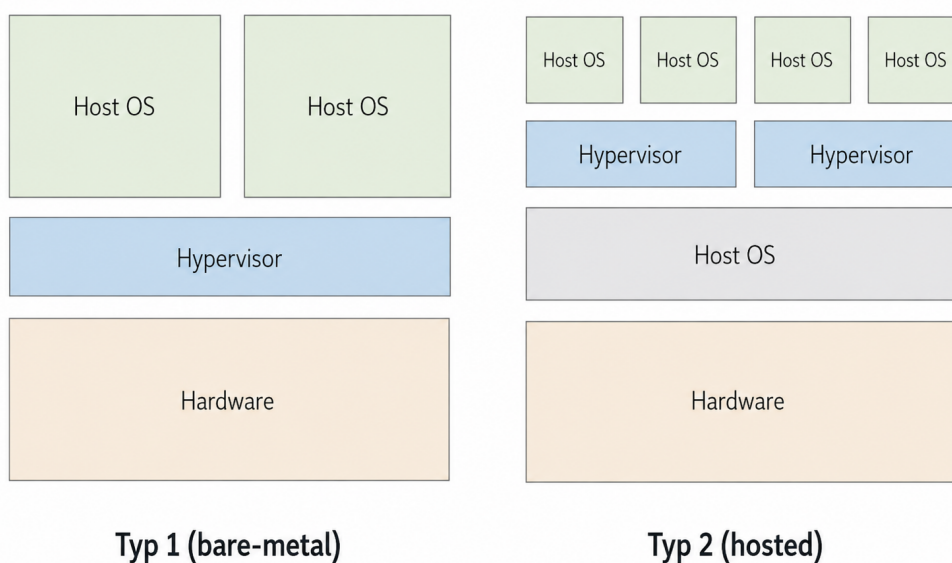
- **Bare-metal**

Také nazývaný Type 1, běží přímo na fyzickém hardwaru hostitelského serveru bez prostřednictví operačního systému. Díky přímému přístupu k hardwarovým prostředkům je velmi efektivní a nabízí vysoký výkon, což z něj činí standardní volbu pro podniková datová centra a cloudová prostředí. Mezi známé zástupce patří VMware ESXi, Microsoft Hyper-V nebo KVM (Kernel-based Virtual Machine), který je součástí jádra Linuxu. [14]

- **Hosted**

Také nazývaný Type 2, instaluje se jako běžná softwarová aplikace na již existující operační systém (tzv. hostitelský OS). Hardwarové zdroje jsou virtuálním strojům přidělovány prostřednictvím tohoto hostitelského systému, což zvyšuje režii a mírně snižuje výkon. Tento typ se nejčastěji využívá na osobních počítačích pro vývoj, testování nebo výuku. Typickými příklady jsou Oracle VirtualBox nebo VMware Workstation.

Infrastrukturou může připomínat kontejner, protože i on potřebuje ke svému běhu hostitelský operační systém. Zásadní rozdíl však spočívá v úrovni virtualizace: zatímco kontejner sdílí jádro hostitelského OS a izoluje pouze uživatelský prostor, Type 2 hypervizor virtualizuje celý hardware a umožňuje běh úplně samostatných operačních systémů s vlastními jádry. [14]



Obrázek 2.10.: Porovnání Typu 1 a Typu 2 hypervizorů (zdroj: vlastní zpracování)

Zástupce kontejneru - Docker

Docker je open-source platforma, která v roce 2013 výrazně popularizovala technologii kontejnerizace a stala se de facto standardem pro balení, distribuci a spouštění aplikačních kontejnerů. Jádrem ekosystému je Docker Engine, démon⁵ běžící na hostitelském operačním systému, který využívá funkcí jádra Linuxu k vytváření a správě izolovaných kontejnerů. Aplikace je definována deklarativně v souboru nazvaném Dockerfile, který popisuje základní obraz (base image), závislosti, konfiguraci prostředí a způsob spuštění aplikace. Výsledný obraz je poté přenosný a může být dále distribuován, přičemž nejznámějším veřejným registrem je Docker Hub. [15]

Zástupce bare-metal - LXD

LXD je moderní nástroj pro správu systémových kontejnerů a virtuálních strojů. LXD umožňuje provozovat kompletní linuxové systémy v izolovaných prostředích. Podporuje širokou škálu linuxových distribucí a nabízí jednotné rozhraní pro jejich spouštění a správu. LXD je navržen tak, aby byl použitelný jak na jediném počítači, tak v rozsáhlých clusterech v datových centrech. Díky tomu se hodí jak pro vývojářská prostředí, tak pro produkční nasazení. Mezi jeho praktické funkce patří například snímky stavu kontejnerů, přesun běžících instancí mezi servery nebo integrovaná správa úložiště a sítě. [16]

LXC jako základní technologie

LXD interně staví na technologii LXC (Linux Containers), která přímo využívá funkcí linuxového jádra pro izolaci procesů. Zatímco LXC poskytuje nízkoúrovňové nástroje pro vytváření kontejnerů, LXD tyto možnosti obaluje do přehledného REST API a přidává pokročilé funkce pro správu. V praxi to znamená, že LXD neimplementuje vlastní mechanismy virtualizace, ale rozšiřuje osvědčené základy LXC o moderní způsob ovládání a orchestrace. [17]

Zástupce hosted - VirtualBox

Oracle VM VirtualBox je multiplatformní virtualizační software, který umožňuje provoz více virtuálních strojů s různými operačními systémy (hostovanými systémy) na jednom fyzickém počítači. Podporuje širokou škálu hostitelských platform včetně Windows, macOS a Linux, což z něj činí univerzální nástroj pro vývojáře, testery i výukové účely. Mezi klíčové funkce patří podpora snapshotů (uložení stavu virtuálního stroje pro pozdější návrat), virtuální síť různých typů (NAT, bridge, internal network) a dynamické přidělování úložného prostoru. I přes praktickou všestrannost je třeba mít na zřeteli, že VirtualBox běží jako aplikace nad hostitelským OS, čímž vzniká dodatečná výkonnostní režie při přístupu k hardwarovým prostředkům, zejména v oblasti grafického výstupu a síťové komunikace. [18]

⁵*Démon (Daemon)* - program běžící na pozadí OS, který automaticky provádí určité úlohy nebo poskytuje služby bez přímé interakce uživatele

Automatizace přípravy prostředí

Ruční vytváření takových prostředí je časově náročné a s rostoucím počtem testovaných systémů také náchylné k chybám. Z tohoto důvodu se v současnosti právě běžně využívají technologie virtualizace a kontejnerizace společně s nástroji pro automatizaci konfigurace. Ty umožňují připravit nové testovací prostředí během několika minut a zajistit, že bude nakonfigurováno stejným způsobem při každém spuštění testu.

Důležitou součástí tohoto přístupu je také koncept Infrastructure as Code, kdy je konfigurace infrastruktury popsána pomocí textových souborů a může být verzována stejně jako zdrojový kód aplikace. Díky tomu lze celý proces nasazení jednoduše opakovat, upravovat a sdílet mezi různými prostředími.

Agent a agent-less přístup

Při automatizaci přípravy prostředí lze rozlišit dva základní přístupy ke správě cílových uzlů: *agent-based* (založený na agentech) a *agent-less* (bez agentů). Volba mezi nimi závisí na charakteru spravované infrastruktury, požadavcích na spolehlivost a frekvenci změn konfigurace.

- **Agent-based přístup**

Tento přístup vyžaduje instalaci speciálního softwaru, tzv. agenta, na každý cílový uzel. Agent pravidelně komunikuje s centrálním řídicím serverem, přijímá deklarativní definici požadovaného stavu systému a kontinuálně zajišťuje, že skutečná konfigurace uzlu odpovídá tomuto definovanému stavu. Díky lokálně běžícímu procesu dokáže agent automaticky detekovat a napravit změny v konfiguraci i během výpadku síťového spojení, neboť není závislý na permanentní dostupnosti řídicího serveru. [19]

Tento model je proto především vhodný pro dlouhodobě existující systémy v produkčním prostředí, kde je klíčová spolehlivost. Nevýhodou je vyšší počáteční režie spojená s nasazením, aktualizací a zabezpečením agentů na každém cílovém zařízení. [19]

- **Agent-less přístup**

Agent-less přístup nevyžaduje instalaci žádného permanentního softwaru na cílových uzlech. Řídicí stroj se k nim připojí standardními bezpečnými protokoly, typicky SSH⁶, přenesení dočasný skript či modul, který se vykoná lokálně na cílovém uzlu, a poté se spojení ukončí. [19]

Tento model výrazně zjednodušuje počáteční nasazení a snižuje režii na cílových uzlech, protože není nutné spravovat žádnou stálou infrastrukturu agentů. Na druhou stranu neposkytuje průběžné sledování stavu. Konfigurace se aplikuje pouze na vyžádání při explicitním spuštění, a pokud dojde k nežádoucím změnám, neexistuje agent, který by je automaticky detekoval a napravit. Agent-less nástroje proto často používají task-based model, kdy je postupně vykonávána předem definovaná posloupnost kroků, nikoli deklarativní popis cílového stavu. [19]

Kritérium	Agent-based	Agent-less
Nasazení	Instalace a registrace agenta na každý uzel	Žádná předchozí příprava uzlu není nutná
Režie na uzlu	Stálý běh agenta (CPU, paměť, síťová komunikace)	Pouze dočasná během provádění úlohy
Model správy	Definovaný cílový stav	Seznam úloh
Spolehlivost při výpadku	Lokální stav, následná synchronizace	Vyžaduje síťové spojení a přístupová práva

Tabulka 2.3.: Srovnání agent-based a agent-less přístupu (zdroj: vlastní zpracování)

Dostupné nástroje ovšem nejsou striktně omezeny na jeden z těchto přístupů. Velmi často různě kombinují obě strategie, aby vyhověly specifickým potřebám spravované infrastruktury.

Jako hlavní představitele agent-based nástrojů lze uvést Puppet, zatímco mezi agent-less nástroje patří Ansible. Přestože se jednotlivé nástroje liší svou architekturou a způsobem komunikace s cílovými uzly, sdílejí několik společných principů, které tvoří základ moderní automatizace konfigurací:

- **Idempotence**

Opakované spuštění stejné konfigurace by mělo vést ke stejnému výsledku, aniž by docházelo k dalším změnám, pokud je systém již v požadovaném stavu. Tento princip zajišťuje bezpečnost a předvídatelnost automatizace. Ve specifických případech, například při generování náhodných hodnot nebo práci s časově závislými daty, však nelze idempotenci vždy zachovat. [20]

- **Deklarativní přístup**

Většina moderních nástrojů umožňuje definovat požadovaný výsledný stav systému namísto přesného popisu jednotlivých kroků vedoucích k jeho dosažení. Samotný nástroj následně určí, jaké operace je třeba provést, aby byl požadovaný stav splněn. Většina nástrojů se snaží tohoto principu držet kvůli přívětivosti pro uživatele, ale většinou podporují i imperativní přístup, který je vhodný pro složitější scénáře, kde je nutné přesně definovat pořadí operací. [20]

- **Čitelnost konfigurace**

Konfigurace jsou navrženy tak, aby byly snadno čitelné a udržitelné i při správě rozsáhlých infrastruktur. Jednotlivé nástroje využívají různé jazyky pro zápis konfigurace, jejichž společným cílem je zpřehlednění správy infrastruktury. [20]

⁶SSH - Secure Shell, protokol pro bezpečnou komunikaci mezi uzly.

Agent-based zástupce - Puppet

Puppet je open-source⁷ nástroj pro správu konfigurací a automatizaci infrastruktury, který byl vytvořen v roce 2005 Lukem Kaniesem. Jeho cílem bylo nabídnout správcům systémů nástroj pro automatizovanou správu rozsáhlých serverových prostředí, kde manuální konfigurace přestává být udržitelná. [21]

V roce 2011 uvedla společnost Puppet Labs na trh komerční edici Puppet Enterprise, která rozšiřuje open-source základ o pokročilé funkce pro řízení přístupu, reporting a orchestraci. V dubnu 2022 byla společnost Puppet získána firmou Perforce, pod jejímž vedením pokračuje další vývoj jak komerční, tak komunitní verze nástroje.

Puppet představuje typický příklad agent-based nástroje. Jeho architektura vychází z modelu klient-server, kde centrální uzel, tzv. *Puppet Server*, spravuje konfigurace a na jednotlivých cílových uzlech běží Puppet Agent. Agent pravidelně, typicky v intervalu 30 minut, iniciuje zabezpečené spojení s Puppet Serverem prostřednictvím protokolu HTTPS. Pomocí vestavěného nástroje Facter nejprve shromáždí aktuální systémové informace (tzv. fakta), které odešle serveru. Na základě těchto faktů, manifestů a modulů server zkompileje tzv. katalog. Specifický plán pro daný uzel definující požadovaný stav systému. Agent následně tento katalog lokálně aplikuje a odesílá report o provedených změnách zpět na server. [21]

Agent-less zástupce - Ansible

Ansible je open-source nástroj pro automatizaci IT úloh, jako je konfigurace systémů, nasazení aplikací a správa infrastruktury. Byl vytvořen v roce 2012 Michaellem DeHaanem, autorem nástrojů Cobbler a Func, s cílem nabídnout jednodušší a přístupnější alternativu k tehdejším nástrojům, které vyžadovaly složitější architekturu a používání agentů, a zároveň poskytnout lepší řešení než jednorázové skripty pro automatizaci úloh.

V roce 2015 byla společnost Ansible získána firmou Red Hat, pod jejímž vedením pokračoval další vývoj nástroje. Následně se Ansible stal součástí platformy Red Hat Automation a jeho využití se rozšířilo zejména v podnikovém prostředí. [20]

Ansible patří do skupiny agent-less nástrojů a s cílovými uzly komunikuje typicky, nikoli však výhradně, prostřednictvím protokolu SSH, což výrazně zjednodušuje jeho nasazení i následnou údržbu. Na cílový uzel se připojuje přes určitý protokol a nahraje tam dočasný program, nazvaný **Ansible module**, který vykoná požadované úlohy a poté se sám odstraní. [20]

⁷Open-source – software, jehož zdrojový kód je volně dostupný uživatelům k prohlížení, úpravám a distribuci.

Monitorování

Aby bylo možné jednotlivé databázové systémy mezi sebou objektivně porovnat, je nutné během testování sbírat nejen výsledky samotných dotazů, ale i informace o tom, jak systém zatěžuje dostupné hardwarové prostředky. Z tohoto důvodu je nedílnou součástí testovacího prostředí monitorovací vrstva, která umožňuje sledovat metriky jako využití CPU, paměti nebo diskových I/O operací v průběhu jednotlivých testů.

Při návrhu monitorovací vrstvy je nutné zvolit způsob, jakým budou metriky získávány z cílových systémů. V praxi se rozlišují dva základní přístupy: *push* a *pull*.

Push styl

V push modelu aktivně odesílají monitorované uzly naměřené metriky do centrálního kolektoru. Cílový systém tedy zná adresu monitorovacího serveru a periodicky či při události posílá data například přes protokoly HTTP, TCP nebo UDP. Tento přístup je vhodný pro dynamická a krátkodobá prostředí, kde se instance často vytvářejí a ruší, nebo pro scénáře s vysokou frekvencí událostí. Nevýhodou je vyšší komplexita na straně monitorovaného uzlu, který musí znát konfiguraci kolektoru a zvládat odesílání i při jeho dočasné nedostupnosti.

Telegraf je open-source agent pro sběr a odesílání metrik vyvinutý společností InfluxData a také typický příklad stylu push. Běží přímo na monitorovaném uzlu jako lehká služba, periodicky čte hodnoty ze systémových zdrojů a aktivně je posílá do jednoho či více cílových úložišť.

Architektura Telegrafu je založena na pluginech: pomocí vstupních pluginů (inputs) lze sbírat data z operačního systému, databází, kontejnerových runtime nebo vlastních aplikací, zatímco výstupní pluginy (outputs) určují, kam se data odesílají. Telegraf podporuje širokou škálu výstupních formátů a protokolů, například přímo do InfluxDB, přes HTTP do Prometheus, do Kafka nebo do souborů. [22]

Pull styl

V pull modelu naopak centrální monitorovací server aktivně dotazuje jednotlivé cílové uzly na aktuální hodnoty metrik. Monitorovaný systém pouze vystaví endpoint (např. HTTP), ze kterého si kolektor data stahuje v definovaném intervalu. Tento přístup usnadňuje správu seznamu sledovaných cílů, umožňuje jednoduše ověřit dostupnost jednotlivých uzlů a centralizuje řízení frekvence sběru. Je obzvláště vhodný pro stabilní prostředí s dlouhodobě běžícími službami. Na druhou stranu vyžaduje, aby byl cílový uzel síťově dostupný z pohledu kolektoru a aby byl schopen metriky vystavovat.

Zástupcem stylu pull je Prometheus, open-source systém pro monitorování a upozornění, který je navržen pro sběr a ukládání časových řad dat. Používá vlastní dotazovací jazyk PromQL pro analýzu a vizualizaci dat, což umožňuje uživatelům vytvářet komplexní dotazy a získávat hluboký náhled z monitorovaných systémů. [23]

Testování

Testování výkonu a spolehlivosti softwarových systémů představuje nedílnou součást návrhu, vývoje a provozu moderních aplikací. Jeho hlavním cílem je systematicky ověřit chování systému při různých úrovních a typech zátěže, identifikovat úzká hrdla, vyhodnotit stabilitu a zajistit, aby systém splňoval požadavky kladené reálným provozem.

V testování můžeme definovat několik klíčových odvětví:

- **Spolehlivost**

Typickými příklady spolehlivosti jsou:

- aplikace funguje tak, jak uživatel očekává
- dokáže tolerovat chyby uživatele
- výkon je dostatečně dobrý pro vyžadované použití, pod očekávaným zatížením
- dokáže zabránit neoprávněnému přístupu a využívání

Pokud jsou všechny tyto body splněny, můžeme říci, že systém je spolehlivý. [3]

Kvalitním nástrojem jak testovat spolehlivost jsou například chaos testy, které záměrně způsobují poruchy v systému, aby se ověřilo, zda dokáže správně reagovat a zotavit se z nich. Jedním takovým nástrojem je Chaos Monkey od Netflixu, který náhodně vypíná instance v produkčním prostředí, aby otestoval odolnost systému vůči selhání. [24]

- **Škálovatelnost**

Škálovatelnost se týká schopnosti systému růst nebo zmenšovat své zdroje v závislosti na zátěži. Systém musí být schopen efektivně zpracovávat rostoucí množství požadavků bez výrazného snížení výkonu.

Zátěž je ovšem ale více než jen počet požadavků. Může zahrnovat i různé typy zátěže, jako jsou například:

- dotazy za sekundu
- poměr čtení a zápisu
- současní aktivní uživatelé

Také je důležité identifikovat, kde se nacházejí úzká hrdla systému, která mohou omezovat jeho škálovatelnost. V jednom systému mohou být problém celkové průměrné zvednutí uživatelů, ale v jiném systému může být extrémní nárůst uživatelů v určité období - odlehlé hodnoty. [3]

- **Výkon**

V oblasti výkonu jsou dvě klíčové otázky, které je třeba zodpovědět:

- Když se zvedne zátěž, ale systémové zdroje zůstávají stejné, jak to ovlivní výkon?
- Když se zvedne zátěž, o kolik musíme zvednout systémové zdroje, aby výkon zůstal stejný?

V systémech které zpracovávají data se často bavíme o propustnosti, která udává, kolik dat je systém schopen zpracovat za jednotku času. V online systémech zase o výkonu často mluvíme v souvislosti s latencí, která udává, jak dlouho trvá zpracování jednotlivého požadavku. [3]

Databázových systémů se týkají všechny tyto aspekty, a proto je důležité je testovat a porovnávat, aby bylo možné vybrat ten nejvhodnější pro konkrétní použití.

Benchmarking a porovnávání systémů

Benchmarking je metodický přístup k porovnávání výkonnosti různých systémů za definovaných a reprodukovatelných podmínek. Cílem je zajistit objektivní srovnatelnost výsledků a eliminovat vlivy okolního prostředí, které by mohly měření zkreslit.

V oblasti databázových systémů se benchmarking využívá například pro porovnání různých datových struktur a úložných strategií, jako dříve zmíněné B-Tree a LSM stromy. [3]

Klíčovým faktorem benchmarkingu je definice správné zátěže, která musí aspoň odpovídat reálnému použití systému. Nevhodně zvolený scénář může vést k výsledkům, které neodrážejí skutečné chování.

Typy benchmarků

Benchmarky lze obecně rozdělit do dvou základních kategorií podle povahy zátěže:

- **Syntetické benchmarky**

Syntetické benchmarky používají uměle vytvořené pracovní zátěže, které simulují typické operace v dané doméně. Jejich výhodou je vysoká reprodukovatelnost, kontrolovatelnost parametrů a možnost zaměřit se na konkrétní aspekty výkonu (např. čtení vs. zápis, sekvenční vs. náhodný přístup). Nevýhodou je, že nemusí plně odrážet složitost reálného provozu. Příkladem může být je Yahoo! Cloud Serving Benchmark (YCSB) [25], který nabízí několik předdefinovaných pracovních sad (workloadů) s různým poměrem operací čtení a zápisu. [26]

- **Aplikační (reálné) benchmarky**

Aplikační benchmarky vycházejí z reálných uživatelských scénářů a datových sad. Poskytují realističtější obraz o chování systému v produkčním prostředí, ale jejich příprava je náročnější a reprodukovatelnost může být nižší. Typickými zástupci mohou být benchmarky definované organizací TPC (Transaction Processing Performance Council), například TPC-C pro zpracování transakcí v reálném čase nebo TPC-H pro rozhodovací podporu (analytické dotazy). [27]

Metody srovnání a měření

Aby byly výsledky benchmarků srovnatelné a věrohodné, je nutné dodržovat systematickou metodologii:

1. Definice cílů a metrik

Před samotným měřením je nutné jednoznačně definovat, co se porovnává a jakými metrikami bude výkon vyjádřen. Pro databázové systémy se nejčastěji používají:

- *Propustnost (throughput)* – počet operací za jednotku času (např. transakce za sekundu, dotazy za sekundu).
- *Latence (response time)* – doba mezi odesláním požadavku a obdržení odpovědi, často vyjadřovaná jako průměr, medián nebo percentily (p50, p95, p99).
- *Využití systémových zdrojů* – spotřeba CPU, operační paměti, diskové I/O operace za sekundu (IOPS) a využití síťového rozhraní.

2. Příprava prostředí

Testovací prostředí musí být izolované od jiných běžících procesů, aby nedocházelo k rušivým vlivům, tzv. (*interferenci*). Důležité je také dokumentovat konfiguraci hardwaru, operačního systému, verzi databázového systému a jeho parametry (např. velikost cache, počet vláken). [28]

3. Opakované měření a statistické zpracování

Jednotlivé běhy benchmarku mohou vykazovat variabilitu způsobenou například garbage collectorem, operacemi systému nebo fluktuacemi teploty CPU (turbo boost). Proto je nutné měření opakovat dostatečný početkrát (typicky 5–10×) a výsledky statisticky zpracovat – uvádět průměr, směrodatnou odchylku a případně odstranit odlehlé hodnoty. [28]

3. Praktická část

3.1. Popis dat

Poskytovatelem dat je Datové centrum Ústeckého kraje (DCUK), které spravuje a zpřístupňuje data z různých systémů veřejné správy. Jedná se o záznamy ze systému pro sledování pohybu vozidel správy a údržby silnic.

Samotný dataset má velikost přibližně 650 MB a obsahuje zhruba 600 tisíc záznamů. Data jsou uložena v tabulkové struktuře ve formátu CSV. Obsahuje celkem 54 atributů různých datových typů. Převládají celočíselné a desetinné hodnoty, které reprezentují měřené nebo stavové veličiny, dále textové atributy sloužící především jako identifikátory a časové údaje. Struktura dat je převážně plochá, přičemž každý řádek představuje jeden záznam o stavu vozidla v konkrétním časovém okamžiku.

Tyto informace umožňují analýzu pohybu vozidel v čase a prostoru, například pro vyhodnocení efektivity údržby silniční sítě nebo sledování tras jednotlivých vozidel.

3.2. Předběžná srovnávací analýza systémů

Pro účely experimentálního porovnání bylo vybráno osm databázových systémů reprezentujících různé přístupy k ukládání a zpracování dat. Výběr zahrnuje relační, sloupcové, dokumentové i specializované databáze pro časové řady, tak i databáze, které nejsou přímo určeny pro ukládání časových řad. Zastoupeny jsou systémy optimalizované pro transakční zpracování (OLTP), analytické zpracování (OLAP) i databáze kombinující vlastnosti obou přístupů.

DB	Kategorie	Typ	Storage	Jazyk	Licence
ClickHouse	Sloupcová	OLAP	MergeTree	SQL	Apache 2.0
InfluxDB 3	TSDB	Hybrid	Parquet + Object store	SQL	OSS / Enterprise
IoTDB	TSDB	Hybrid	TsFile	IoTDB-SQL	Apache 2.0
MariaDB	RDBMS	OLTP	InnoDB, Aria, MyRocks	SQL	GPLv2
MariaDB ColumnStore	Sloupcová	OLAP	Sloupcové	SQL	GPL
MongoDB	NoSQL	OLTP	WiredTiger	MQL	SSPL
QuestDB	TSDB	Hybrid	Sloupcový	SQL	Apache 2.0
TimescaleDB	TSDB/RDBMS	Hybrid	Hypertables	SQL	Apache 2.0

Tabulka 3.1.: Srovnání databázových systémů

ClickHouse

ClickHouse¹ je sloupcově orientovaná databáze určená pro analytické dotazy v reálném čase (OLAP), původně vyvinutá společností Yandex jako interní nástroj pro zpracování velkých objemů dat. Později byla uvolněna jako open-source projekt a postupně se stala jednou z populárních databází pro analytické zpracování dat. [29]

Hlavním důvodem vzniku ClickHouse byla potřeba efektivního zpracování a analýzy rozsáhlých logových a telemetrických dat v reálném čase, kde tradiční relační databáze nedokázaly splnit požadavky na výkon.

ClickHouse je dostupný jak ve formě open-source řešení pro self-hosting, tak i jako plně spravovaná cloudová služba. Open-source varianta umožňuje nasazení na vlastních serverech nebo v libovolném cloudovém prostředí.

Z pohledu ukládání dat využívá ClickHouse vlastní rodinu úložných systémů založených na technologii *MergeTree*. Ta je koncepčně podobná struktuře LSM Tree popsané v kapitole 2.3. [30]

Databáze podporuje standardní jazyk SQL rozšířený o analytické a časové funkce. Dále podporuje jazyky jako jsou PRQL (Pipelined Relational Query Language), nebo KQL (Kusto Query Language). [30]

Databáze je licencovaná pod Apache License 2.0², což umožňuje široké využití a přizpůsobení v různých projektech a prostředích.

InfluxDB 3

InfluxDB 3³ je časově orientovaná databáze navržená pro ukládání a analýzu dat závislých na čase, například metrik, senzorických dat nebo logů.

¹<https://clickhouse.com>

²<https://www.apache.org/licenses/LICENSE-2.0>

³<https://www.influxdata.com>

Vznikla v roce 2013 společností InfluxData. Je optimalizována pro vysokou rychlost zápisu a práci s časovými intervaly, což ji činí vhodnou pro IoT, monitoring a observabilitu systémů. [31]

InfluxDB nyní využívá hlavně jazyk SQL, v minulosti ovšem ale využíval vlastní dotazovací jazyky InfluxQL a následně Flux. V nejnovějších verzích podpora pro Flux už není. Kromě toho je dostupný také HTTP API⁴ přístup. [31]

Nejnovější verze InfluxDB už nevyužívá tradiční datové struktury, ale má vlastní úložný systém ve formátu mikroslužeb [31]:

- **Ingester**

Zpracovává příchozí zápisy, drží nejčerstvější data v operační paměti a pomocí transakčního logu⁵ zajišťuje jejich bezpečnost. Následně se stará o jejich přesunutí do trvalého úložiště.

- **Object Store**

Úložiště používá soubory ve formátu Apache Parquet, které jsou optimalizované pro analytické dotazy a umožňují efektivní kompresi a ukládání velkých objemů dat. [33]

- **Compactor**

Proces, který na pozadí slučuje Parquet soubory do větších celků.

InfluxDB je optimalizována především pro write-heavy zátěž, tedy scénáře s velmi vysokou frekvencí zápisu časově označených dat. Naopak komplexní analytické dotazy nad velmi rozsáhlými historickými daty mohou být méně efektivní.

Z pohledu ekosystému je InfluxDB často využívána společně s nástroji Prometheus a Grafana, pro které existuje oficiální datový zdrojový plugin umožňující přímé napojení a vizualizaci dat.

InfluxDB je dostupná ve více edicích, od open-source InfluxDB 3 Core až po placené varianty jako InfluxDB 3 Enterprise a cloudové služby. Pokročilé funkce, například vysoká dostupnost, clustering⁶ nebo produkční škálování, jsou typicky součástí Enterprise nebo cloudových verzí, zatímco Core je určený pro single-node a neprodukční použití. Licenční a funkční rozsah se liší podle zvolené edice, nicméně open-source varianta InfluxDB 3 Core je dostupná pod licencí MIT⁷ nebo Apache License 2.0⁸.

Apache IoTDB

Apache IoTDB⁹ je časově orientovaná databáze navržena specificky pro správu velkých objemů časových řad generovaných v prostředí Internetu věcí (IoT) a průmyslového internetu věcí (IIoT). Projekt vznikl na univerzitě Tsinghua v Pekingu pod vedením profesora Jianmina Wanga, kde

⁴API - Application Programming Interface, rozhraní, přes které spolu komunikují různé aplikace.

⁵Write-ahead Log (WAL) - technika, kdy se každá změna nejdříve trvale zapisuje do logu a teprve potom se aplikuje do datových souborů, aby šlo po pádu systém obnovit přehráním logů. [32]

⁶Clustering - technika, která seskupuje záznamy podle jejich podobnosti nebo klíče, aby se fyzicky ukládaly blízko sebe a tím se zrychlilo jejich následné vyhledávání.

⁷<https://opensource.org/licenses/MIT>

⁸<https://www.apache.org/licenses/LICENSE-2.0>

⁹<https://iotdb.apache.org>

tým od roku 2011 řešil problémy s ukládáním masivních strojových dat a v roce 2016 formálně představil vlastní formát TsFile a na jeho základě vyvinul databázi IoTDB. [34]

Hlavním důvodem vzniku IoTDB byla neschopnost existujících NoSQL i relačních databází efektivně zpracovávat specifika průmyslových časových řad: extrémně vysokou frekvenci zápisu z milionů senzorů, potřebu vysoké komprese při zachování přesnosti, zpožděná data a požadavek na analytiku přímo na okrajových zařízeních. [34]

Z pohledu ukládání dat využívá IoTDB vlastní sloupcový formát TsFile, optimalizovaný pro časové řady. Data jsou nejprve zapisována do paměťové struktury (*memtable*) a zajištěna transakčním logem (WAL). Následně jsou převedena do neměnných souborů TsFile na disku. Celková organizace vychází z LSM-based přístupu, který je koncepčně podobný struktuře popsané v kapitole 2.3. Díky specifickým kompresním a kódovacím technikám pro časové řady dosahuje IoTDB vysokých kompresních poměrů při zachování rychlého zápisu i čtení. [34]

Systém je architektonicky rozdělen do dvou hlavních komponent:

- **ConfigNode**

Spravuje metadata clusteru¹⁰, schéma časových řad a koordinaci distribuovaných uzlů. Zajišťuje jednotný stav mezi uzly a vysokou dostupnost v režimu clusteru.

- **DataNode**

Zajišťuje vlastní zpracování dotazů a úložiště dat. Každý DataNode uchovává fyzická data ve formátu TsFile a provádí lokální výpočetní operace nad přidělenými časovými řadami.

Architektura umožňuje horizontální škálování přidáváním DataNode, čímž se zvyšuje jak kapacita úložiště, tak propustnost zpracování dotazů.

Databáze podporuje SQL-like dotazovací jazyk (IoTDB-SQL), který rozšiřuje standardní SQL o funkce specifické pro časové řady, jako je LAST dotaz pro okamžité získání nejnovější hodnoty senzoru s mikrosekundovou latencí.

IoTDB je dostupné jako open-source projekt pod licencí Apache 2.0 pro self-hosting, a to jak v samostatném, tak v distribuovaném režimu.

MariaDB

MariaDB¹¹ je open-source relační databázový systém vyvinutý jako komunitní nástupce MySQL. Vznikla v roce 2009 jako reakce na akvizici Sun Microsystems (a s ním i MySQL) společností Oracle. Cílem projektu bylo zachovat svobodnou a otevřenou licenční politiku, zároveň však poskytnout plnou kompatibilitu s existujícími MySQL aplikacemi a postupně rozšiřovat funkcionalitu nad rámec původního systému. [35]

Z architektonického hlediska využívá MariaDB modulární koncept pluggable storage engine, což umožňuje volit optimální způsob ukládání dat podle charakteru úlohy. Vedle standardního InnoDB

¹⁰Cluster - skupina uzlů, které spolupracují a fungují jako jeden logický celek

¹¹<https://mariadb.org>

(který zajišťuje transakční zpracování a vnější klíče) jsou dostupné například Aria pro rychlé tabulky bez nutnosti transakcí, MyRocks založený na technologii RocksDB pro efektivní zápis, nebo Spider pro horizontální dělení dat napříč více servery.

Databázový server zpracovává dotazy v jazyce SQL a podporuje pokročilé konstrukce jako jsou common table expressions (CTE), okénkové funkce, dotazy přes JSON datové typy a geoprostorové indexy. [35]

MariaDB je šířena pod licencí GNU General Public License verze 2¹², což zaručuje svobodné použití, studium i modifikaci zdrojových kódů. Systém je dostupný ve formě komunitní edice MariaDB Community Server, která je vhodná pro většinu produkčních i vývojových scénářů. Pro organizace s požadavkem na profesionální podporu, pokročilé zabezpečení nebo správu existuje placená MariaDB Enterprise.

MariaDB ColumnStore

MariaDB ColumnStore¹³ je sloupcové rozšíření pro relační databázi MariaDB, navržený pro analytické zpracování velkých objemů dat (OLAP) a hybridní transakčně-analytické zpracování (HTAP). Původně vychází z projektu InfiniDB společnosti Calpont, který byl v roce 2014 uvolněn pod open-source licencí a následně integrován do ekosystému MariaDB. [36]

Na rozdíl od samostatných analytických databází je ColumnStore plně integrován do MariaDB Serveru jako plugin storage engine. To umožňuje v rámci jedné instance kombinovat transakční tabulky v InnoDB a analytické tabulky v ColumnStore, případně provádět cross-engine JOINy mezi oběma typy úložišť. [36]

Z pohledu dotazování podporuje ColumnStore standardní SQL včetně agregací, okenních funkcí a komplexních JOINů. Pro rychlý bulk load dat je k dispozici nástroj cpimport, který obchází SQL vrstvu a zapisuje data přímo do sloupcového úložiště, což minimalizuje režii při importu velkých datasetů. [36]

Systém je rozdělen do dvou hlavních typů komponent:

- **User Modules (UM)** Slouží jako vstupní bod systému a zajišťují SQL engine, správu připojení a orchestraci dotazů. User Module přijme SQL dotaz, provede jeho analýzu a rozdělí jej na dílčí operace, které jsou následně distribuovány do výpočetních uzlů. Zároveň může provádět operace, které nelze paralelizovat, a vrací finální výsledky zpět klientovi prostřednictvím MariaDB serveru.
- **Performance Modules (PM)**
Zajišťují samotné distribuované zpracování dat. Každý Performance Module provádí výpočetní operace nad přidělenou částí dat, která je uložena ve sloupcové formě. PM uzly čtou a zapisují data z columnar storage a vracejí mezivýsledky zpět User Module.

¹²<https://www.gnu.org/licenses/old-licenses/gpl-2.0.cs.html>

¹³<https://mariadb.com/docs/analytics/mariadb-columnstore>

Architektura je plně škálovatelná horizontálně, přidáním PM se zvyšuje výkon zpracování dotazů, zatímco přidáním UM se zvyšuje propustnost a dostupnost systému. Každý uzel je navíc vícevláknový, což dále zvyšuje výkon na úrovni jednotlivých serverů.

Z pohledu ukládání dat využívá MariaDB ColumnStore sloupcový model, kde jsou data ukládána po sloupcích namísto řádků. Tento přístup umožňuje číst pouze relevantní sloupce pro daný dotaz, čímž se výrazně snižuje objem načítaných dat. Řádky jsou logicky rekonstruovány pomocí offsetů napříč sloupcovými soubory. Horizontální škálování je dosaženo rozdělením dat mezi PM, zatímco další optimalizace čtení je zajištěna pomocí Extent Map, které obsahují metadatové informace o rozsazích dat a umožňuje přeskočení nerelevantních bloků.

Jakožto modul pro MariaDB také spadá pod licenci GNU General Public License verze 2.

MongoDB

MongoDB¹⁴ je dokumentově orientovaná NoSQL databáze navržená pro ukládání a zpracování velkých objemů nestrukturovaných a polostrukturovaných dat. Vznikla v roce 2009 společností 10gen (dnes MongoDB Inc.) jako odpověď na rostoucí potřebu flexibilního datového modelu, který by lépe reflektoval strukturu moderních aplikací a umožnil horizontální škálování bez komplexních změn schématu. [37]

Hlavním důvodem vzniku MongoDB byla neschopnost relačních databází efektivně zpracovávat rychle se měnící data s proměnlivou strukturou, zejména v kontextu webových aplikací a real-time systémů, kde je požadována vysoká propustnost zápisu a jednoduchá distribuce dat napříč servery. [37]

Z pohledu ukládání dat využívá MongoDB binární reprezentaci JSONu nazývanou BSON¹⁵. Data jsou organizována do kolekcí, které obsahují jednotlivé dokumenty s flexibilním schématem. Každý dokument může mít odlišnou strukturu. Pro trvalé uložení na disk využívá výchozí storage engine WiredTiger¹⁶, která podporuje kompresi, šifrování a řízení souběžnosti na úrovni dokumentů. [37]

Z pohledu dotazování používá MongoDB vlastní dotazovací jazyk MQL (MongoDB Query Language), který umožňuje vyhledávání dokumentů na základě jejich struktury, obsahu i vnořených polí. Kromě základních CRUD¹⁷ operací podporuje komplexní transformace a výpočty přímo v databázi, textové vyhledávání, geoprostorové dotazy a operace nad časovými řadami. [37]

MongoDB je dostupná ve více edicích. Komunitní edice MongoDB Community Server je šířena pod licenci SSPL (Server Side Public License)¹⁸, která umožňuje volné použití a modifikace při splnění určitých podmínek pro cloudové nasazení. Pro komerční použití s pokročilými funkcemi, jako je

¹⁴<https://www.mongodb.com>

¹⁵BSON - Binary JSON, binární formát serializace dokumentů podobný JSON, rozšířený o datové typy jako datum, binární data nebo ObjectId.

¹⁶WiredTiger podporuje jak řádkové, tak sloupcové ukládání dat. Tyto schémata je schopný aplikovat i přímo na úrovni kolekcí. Využívá jak LSM stromy 2.3, tak i B-Tree struktury 2.3. [38]

¹⁷CRUD - Create, Read, Update, Delete, základní operace databázových systémů

¹⁸<https://www.mongodb.com/legal/licensing/server-side-public-license>

in-memory engine, šifrování na úrovni databáze nebo auditování, je určena placená MongoDB Enterprise Edition. Kromě toho je nabízena plně spravovaná cloudová služba MongoDB Atlas, která poskytuje automatizované zálohování, monitoring a globální distribuci dat. [37]

QuestDB

QuestDB¹⁹ je sloupcově orientovaná databáze určená pro časové řady a analytické úlohy v reálném čase, původně vyvinutá jako interní nástroj pro sledování plavidel v energetickém obchodním domě v roce 2012. Později byl projekt přepracován na open-source databázi a oficiálně uveden v roce 2019. Hlavním důvodem vzniku byla potřeba zpracovávat vysokofrekvenční data s mikrosekundovou latencí, kde tradiční databáze nedokázaly zajistit požadovanou propustnost zápisu a rychlost dotazů. [39]

Z pohledu ukládání dat využívá QuestDB sloupcový model založený na relačních tabulkách s typovanými sloupci a označeným časovým razítkem. Příchozí data jsou nejprve zapisována do transakčního logu (WAL). Následně jsou převedena do nativního sloupcového binárního formátu s rozložením souborů podle sloupců a časových partitionů, což umožňuje dotazům číst pouze relevantní sloupce a časové úseky. Dotazy jsou dále optimalizovány pro rychlé vektorizované agregace. Starší data mohou být automaticky přesunuta do úložiště objektů ve formátu Apache Parquet. [39]

QuestDB podporuje standardní SQL rozšířený o časové funkce. Pro vysokorychlostní ingestování je k dispozici protokol InfluxDB Line Protocol, zatímco pro dotazování implementuje PostgreSQL wire protocol, což umožňuje připojení standardními PostgreSQL klienty bez nutnosti speciálních ovladačů. K dispozici je také REST API a vestavěná webová konzole.

QuestDB je dostupná jako open-source projekt pod licencí Apache License 2.0.

TimescaleDB

TimescaleDB²⁰ je open-source relační databázový systém rozšířený o schopnosti pro práci s časovými řadami a analytické zpracování, implementovaný jako rozšíření pro PostgreSQL. Projekt vznikl v roce 2015 z akademického výzkumu na univerzitě Princeton, přičemž první veřejná verze byla uvedena v roce 2017. Hlavním motivem bylo vytvořit systém, který by spojil výhody tradičních relačních databází, plnou podporu SQL, transakční konzistenci a bohatý ekosystém nástrojů, s výkonem specializovaných databází pro časové řady, aniž by bylo nutné opouštět osvědčenou platformu PostgreSQL. [40]

Pro ukládání dat využívá TimescaleDB koncept hypertables, abstrakce nad standardní PostgreSQL tabulkou, která je automaticky rozdělena do menších časových segmentů (chunků), přičemž pro uživatele i aplikace zůstává viditelná jako jediná logická tabulka. Data jsou rozdělována do chunků

¹⁹<https://questdb.com>

²⁰<https://www.tigerdata.com>

na základě časového sloupce a volitelně i prostorového klíče. Proti výpadkům implementuje transakční log (WAL).

Systém podporuje plný standardní SQL prostřednictvím PostgreSQL, rozšířený o časové funkce pro pokročilou analytiku. Klíčovou vlastností jsou continuous aggregates, které automaticky předpočítávají agregace na pozadí a umožňují tak real-time analytiku nad obrovskými datovými sadami. Díky implementaci jako PostgreSQL extension je plně kompatibilní s existujícími PostgreSQL nástroji, ovladači a ekosystémem. [40]

TimescaleDB je šířen pod licencí Apache License 2.0, což umožňuje volné použití i modifikace. Systém je dostupný jako open-source rozšíření pro self-hosting na vlastních serverech. Pro produkční prostředí s požadavkem na automatizované zálohování, monitoring a horizontální škálování je nabízena plně spravovaná cloudová služba Timescale Cloud, dostupná na platformách AWS, Azure a GCP. [40]

3.3. Návrh testovacího scénáře

V následujících odstavcích popíši realizace a návrhy testovacího prostředí, které jsem využil pro porovnání jednotlivých databázových systémů. V mém prostředí chci zadefinovat co nejbližší jednotné prostředí, jako jsou například stejná data a stejné HW prostředky. To budu realizovat pomocí orchestrace založené na Ansible. Monitoring běhu jednotlivých kontejnerů je založen na systému Prometheus a sesbíraná HW data budou vizualizována pomocí nástroje Grafana.

Testovací prostředí

Pro realizaci experimentů byl zvolen operační systém Ubuntu Server v konfiguraci CLI (Command Line Interface) bez grafického prostředí. Důvodem této volby byla především snaha minimalizovat režii operačního systému a zajistit co největší podíl hardwarových prostředků pro samotné databázové systémy. Současně jde o stabilní a široce podporovanou linuxovou distribuci běžně používanou v serverových prostředích.

Ubuntu Server standardně poskytuje virtualizační platformu LXD, která umožňuje vytváření jak systémových kontejnerů, tak virtuálních strojů. Pro účely této práce byly využity kontejnery, protože poskytnutá architektura od DCUK nepodporuje vytváření virtuálních strojů. Díky tomu lze jednotlivé databázové systémy provozovat v navzájem nezávislých prostředích bez rizika vzájemného ovlivňování.

Současně LXD umožňuje rychlé vytváření, konfiguraci i odstraňování kontejnerů, což výrazně usnadňuje opakovatelnost experimentů a zajišťuje, že všechny databázové systémy byly testovány za srovnatelných podmínek.

Automatizace nasazení

Pro automatizaci přípravy testovacího prostředí byl zvolen nástroj Ansible. Hlavním důvodem byla jeho agent-less architektura, která nevyžaduje instalaci žádného trvale běžícího agenta na spravovaných systémech.

Vzhledem k tomu, že jednotlivé kontejnery vznikaly pouze po dobu experimentů a po jejich dokončení byly odstraněny, by instalace a správa agentů představovala zbytečnou režii. Ansible komunikuje prostřednictvím protokolu SSH a konfiguraci provádí pouze při spuštění jednotlivých úloh. Tento přístup umožňuje jednoduše opakovat celé nasazení, zajišťuje reprodukovatelnost experimentů a současně minimalizuje vliv automatizační vrstvy na měřené výsledky.

Další výhodou je možnost popsat kompletní infrastrukturu pomocí Infrastructure as Code, díky čemuž lze všechny experimenty kdykoliv znovu vytvořit ve stejné konfiguraci.

Monitorování

Pro sledování průběhu experimentů byl zvolen monitorovací stack tvořený nástroji Prometheus a Grafana.

Prometheus zajišťuje pravidelný sběr systémových metrik prostřednictvím pull modelu, který dobře odpovídá charakteru testovacího prostředí a vzhledem k tomu, že hlavní komponenta běží na jiném stroji než databázové systémy, přidává menší režii. Během experimentů byly sledovány zejména metriky využití procesoru, operační paměti, diskových operací a dalších systémových prostředků.

Grafana sloužila jako vizualizační vrstva, která umožnila průběžně sledovat chování jednotlivých databázových systémů během testů a současně usnadnila následnou analýzu naměřených dat.

3.4. Konfigurace testovacího prostředí a databázových systémů

Tato kapitola popisuje konfiguraci testovacího prostředí a jednotlivých databázových systémů použitých při experimentech. Cílem bylo zajistit srovnatelné podmínky pro všechny testované databáze a minimalizovat vliv rozdílného nastavení na výsledky měření. Následující podkapitoly se věnují instalaci Ansible, architektuře prostředí a ukázkové konfiguraci databázového systému ClickHouse.

Instalace

Ansible byl nainstalován pouze na řídicím stroji, tedy na notebooku, ze kterého je spouštěna veškerá orchestrace. Díky agent-less architektuře není potřeba instalovat žádnou komponentu

přímo na testovaných kontejnerech, ty vyžadují pouze funkční SSH server a Python 3, což je zajištěno již při jejich vytváření.

Instalace na řídicím stroji s Ubuntu byla provedena pomocí balíčkovacího systému `apt`:

```
sudo apt update
sudo apt install -y ansible
```

Ukázka kódu 3.1: Instalace Ansible

Vzhledem k tomu, že vytváření a správa kontejnerů je realizována přes LXD, bylo nutné doinstalovat kolekci `community.general` (často již ale bývá přímo v balíčku Ansible), která poskytuje modul `lxd_container` používaný v roli `common-create` (viz podsekce 3.4):

```
ansible-galaxy collection install community.general
```

Ukázka kódu 3.2: Instalace kolekce `community.general`

Architektura

Každý testovaný databázový systém běží ve svém samostatném LXD kontejneru, který je nakonfigurován s přidělenými hardwarovými prostředky (CPU, RAM, diskový prostor).

Při vytvoření kontejneru se do něj nainstaluje zvolený databázový systém a Node Exporter pro sběr metrik.

Na vyhrazeném kontejneru běží Grafana a Prometheus, které zajišťují sběr a vizualizaci metrik z jednotlivých kontejnerů.

Ansible Inventář

Inventář definuje cílové hosty a je rozdělen do tří skupin. Pro pilotní i lokální testování je použit soubor `inventory/local`, který odpovídá síti vytvářené LXD na notebooku:

```
databases:
  vars:
    ansible_connection: ssh
  hosts:
    mariadb:
      ansible_host: 10.187.240.2
    questdb:
      ansible_host: 10.187.240.3
    influxdb:
      ansible_host: 10.187.240.4
    iotdb:
      ansible_host: 10.187.240.5
    clickhousedb:
      ansible_host: 10.187.240.6
    timescaledb:
      ansible_host: 10.187.240.7
    columnstoredb:
      ansible_host: 10.187.240.8
    mongodb:
      ansible_host: 10.187.240.9

monitoring:
  hosts:
    grafana:
      ansible_host: 10.187.240.240

servers:
  hosts:
    localhost:
      ansible_host: localhost
      ansible_connection: local
```

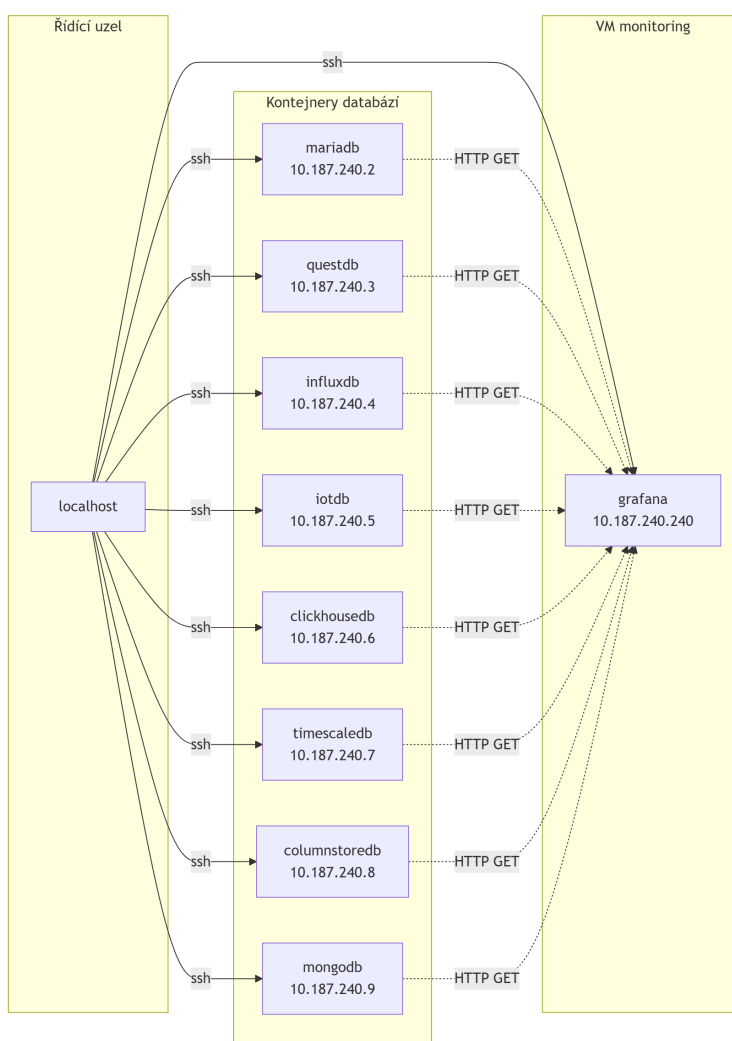
Ukázka kódu 3.3: `inventory/local`

Skupina **databases** obsahuje osm hostů, jeden pro každý testovaný databázový systém, se staticky přidělenou IP adresou uvnitř lokální LXD sítě. Připojení probíhá standardně přes SSH (`ansible_connection: ssh`).

Skupina **monitoring** obsahuje jediného hosta **grafana**, na kterém běží monitorovací stack (Prometheus a Grafana). Tento host je zároveň cílem notifikace po každém testu (reload konfigurace Prometheus, zápis Grafana anotace, viz podsekcce 3.6).

Skupina **servers** obsahuje pouze `localhost` s `ansible_connection: local`. Tato skupina je vstupním bodem pro playbooky `setup.yml` a `test.yml`. Úlohy vytváření a spouštění LXD instancí (modul `lxd_container`) se totiž musí vykonávat lokálně na stroji, kde běží samotný LXD daemon, nikoliv na vzdáleném databázovém hostu.

Pro nasazení na cílovou infrastrukturu poskytovatele dat (DCUK) existuje analogický soubor `inventory/remote`, který má identickou strukturu, ale liší se v adresním rozsahu a v připojení ke skupině **databases** přes SSH ProxyJump na vstupní bránu. Přepnutí mezi prostředím je řešeno buď přímo předáním inventáře při volání `ansible-playbook ... -i <cesta_k_inventari>` nebo parametrem u shell skriptů (například `run_test.sh ... -i <inventory>`), takže není nutné měnit žádný playbook.



Obrázek 3.1.: Ukázka komunikace mezi uzly (zdroj: vlastní zpracování)

Konfigurace databází

Každý databázový systém má vlastní roli ve stejné struktuře: `defaults/main.yml` definuje proměnné, `tasks/main.yml` pouze zřetězí instalaci a konfiguraci, a samotná logika je rozdělena do jednotlivých `tasks/*.yml`. Jako reprezentativní příklad je níže rozebrána role `clickhousedb`.

Proměnné role

Výchozí proměnné v `roles/clickhousedb/defaults/main.yml` definují jak parametry pro vytvoření LXD instance, tak parametry samotné databáze:

```
keyring: "/usr/share/keyrings/clickhouse-keyring.gpg"
import_dir: "/var/lib/clickhouse/user_files"

vm_name: "clickhouse"
vm_packages:
  - apt-transport-https
  - ca-certificates
vm_disk_size: 6GB

data_file: #####

db_host: localhost
db_port: 9000
db_table: car_telemetry

db_key_url: "https://packages.clickhouse.com/rpm/lts/repoata/
            repomd.xml.key"
db_engine: "MergeTree"
```

Ukázka kódu 3.4: Výchozí proměnné role `clickhousedb`

`vm_disk_size` a `vm_packages` jsou čteny rolí `common-create` při vytváření LXD instance, tedy ještě před tím, než se role `clickhousedb` vůbec spustí.

Instalace databáze

Soubor `tasks/install.yml` provede instalaci a základní zprovoznění serveru:

```
- name: Download GPG key
  ansible.builtin.apt_key:
    url: "{{db_key_url}}"
    keyring: "{{keyring}}"
    state: present

- name: Add ClickHouse repository
  ansible.builtin.apt_repository:
    repo: >
      deb [signed-by={{ keyring }} arch=amd64]
      https://packages.clickhouse.com/deb lts main
    filename: clickhouse
    register: repo_result
    until: repo_result is succeeded

- name: Install ClickHouse packages
  ansible.builtin.apt:
    name: [clickhouse-server, clickhouse-client]
    state: present
    update_cache: true
  environment:
    DEBIAN_FRONTEND: noninteractive
  until: result is succeeded

- name: Ensure ClickHouse listens on all interfaces
  ansible.builtin.copy:
    dest: /etc/clickhouse-server/config.d/listen.xml
    content: |
      <yandex>
        <listen_host>0.0.0.0</listen_host>
      </yandex>
  notify: Restart ClickHouse

- name: Enable and start ClickHouse service
  ansible.builtin.service:
    name: clickhouse-server
    enabled: true
    state: started
```

Ukázka kódu 3.5: `roles/clickhousedb/tasks/install.yml` (zkráceno)

Z výpisu uvedeného konfiguračního souboru 3.5 jsou patrné následující bloky:

- **Download GPG key** - stažení GPG²¹ klíče pro ověření balíčků z repozitáře ClickHouse.
- **Add ClickHouse repository** - přidání repozitáře do systému, aby bylo možné instalovat balíčky `clickhouse-server` a `clickhouse-client`.
- **Ensure ClickHouse listens on all interfaces** - přepsání výchozí konfigurace, aby ClickHouse poslouchal na všech síťových rozhraních.
- **Enable and start ClickHouse service** - povolení a spuštění služby ClickHouse.

Výchozí konfigurace ClickHouse poslouchá pouze na loopback rozhraní²², proto je přes `config.d/listen.xml` přepsáno chování na `0.0.0.0`, bez této úpravy by nebylo možné se serverem komunikovat mimo samotný kontejner. Změna konfiguračního souboru je navázána na handler, který po jejím zápisu ClickHouse restartuje:

```
- name: Restart ClickHouse
  ansible.builtin.service:
    name: clickhouse-server
    state: restarted
```

Ukázka kódu 3.6: `roles/clickhousedb/handlers/main.yml`

Vytvoření datové struktury

Po instalaci vytvoří `tasks/setup_database.yml` cílovou tabulku pomocí příkazového klienta `clickhouse-client`. Tabulka odpovídá struktuře testovaného datasetu (viz sekce 3.1) a je definována s ohledem na typický přístup k datům, dotazy filtrující podle vozidla a času:

```
CREATE TABLE IF NOT EXISTS {{ db_table }}
(
    orig_time      DateTime64(9, 'Europe/Prague'),
    GpsTime        DateTime64(9, 'Europe/Prague'),
    VehicleID      String,
    Longitude       Decimal(14,10),
    Latitude        Decimal(14,10),
    SpeedGps        Decimal32(3),
    ...
)
ENGINE = {{ db_engine }}
ORDER BY (VehicleID, GpsTime)
PARTITION BY toDate(GpsTime)
PRIMARY KEY (VehicleID, GpsTime)
```

Ukázka kódu 3.7: Výřez z `roles/clickhousedb/tasks/setup_database.yml`

²¹ GPG Key - kryptografický klíč používaný k ověření identity a podepisování či šifrování dat

²² Loopback rozhraní - síťové rozhraní, které umožňuje komunikaci pouze v rámci stejného zařízení

Klíč řazení (`ORDER BY`) a primární klíč jsou zvoleny jako dvojice (`VehicleID`, `GpsTime`), díky čemuž jsou data uvnitř jednotlivých částí fyzicky uspořádána podle vozidla a času. Členění podle dne (`toDate(GpsTime)`) zároveň umožňuje ClickHouse při dotazech s časovým omezením přeskočit celé sekce, které se v daném rozsahu nevyskytují.

Role `clickhousedb` tímto nijak nezasahuje do vytváření samotného virtuálního stroje, to má na starosti obecná role `common-create`, která nejprve deleguje výběr hardwarového profilu na roli `profile-loader`, poté vytvoří LXD instanci a nakonec vyčká, než bude cíl dostupný přes SSH. Role databáze se tak spouští až nad již běžícím a připraveným systémem, což umožňuje stejný postup jednotně opakovat pro všech osm testovaných systémů, liší se ovšem jednotlivé kroky při instalaci a vytvoření databází.

3.5. Pilotní nasazení systémů

Cílem pilotního nasazení nebylo měřit výkon, ale ověřit, že je celý navržený postup, vytvoření kontejneru, instalace databáze, import datasetu a spuštění testovacích dotazů, funkční v celé své šíři, dříve než bude spuštěn na cílové infrastruktuře poskytovatele dat.

Vzhledem k omezené kapacitě notebooku nebylo možné provozovat všech osm databázových kontejnerů současně. Pilotní testování proto probíhalo postupně, vždy s jedním běžícím databázovým strojem, přičemž ostatní byly po dobu testu vypnuty (role `common-stop`), stejný princip následně převzal i produkční skript `run_test.sh` (sekce 3.6).

Pilotní nasazení proběhlo ve dvou fázích:

- **Manuální ověření**

V první fázi byl celý postup pro ClickHouse odzkoušen manuálně, přímými příkazy nad LXD (`lxc launch`, `lxc exec`) a databázovým klientem. Instalace balíčků, přidání repozitáře, vytvoření testovací tabulky, import datového souboru a ověřovací agregační dotaz (`SELECT sum(SumSalt) . . .`) sloužily k potvrzení, že zvolený formát dat, schéma tabulky i způsob importu odpovídají očekávání, než byly zakomponovány do Ansible role.

- **Automatizované ověření**

Po manuálním ověření byl stejný postup přepsán do role `clickhousedb` a spuštěn přes `ansible-playbook`. Kontejner byl mezi jednotlivými pokusy vždy odstraněn (`lxc delete <database>`) a znovu vytvořen automatizovaně, čímž bylo potvrzeno, že celý proces je opakovatelný a nezávislý na předchozím stavu systému.

Tento dvoufázový přístup byl následně zopakován i pro zbývající databázové systémy a stal se šablonou pro strukturu všech rolí v adresáři `roles/` – nejprve manuální ověření instalačního postupu dané databáze, poté jeho přenesení do role. Výstupem pilotní fáze tedy nebyla naměřená data, ale funkční a opakovatelná automatizace, na jejímž základě mohlo být spuštěno systematické testování popsané v následující sekci.

3.6. Implementace testů

Pro testování databázových systémů jsem vytvořil následující sadu testů:

- **Insert test**

Test zaměřený na měření rychlosti zápisu dat do databáze.

- **Single point test**

Test zaměřený na měření rychlosti vrácení jednoho specifického záznamu z databáze.

- **Condition test**

Test zaměřený na měření rychlosti vrácení všech záznamů splňujících určitou podmínku. Jako podmínku jsem si zvolil vrácení všech záznamů, které mají polohu v určitém geografickém rozsahu za určité období.

- **Aggregate test**

Test zaměřený na měření rychlosti provedení agregačního dotazu nad databází. V agregačním testu jsem zvolil dotaz, který vrací průměrnou rychlost, maximální rychlost a počet záznamů pro každé vozidlo za určité období.

Všechny tyto testy se spouští pomocí skriptu `run_test.sh`, který přijímá parametry pro výběr databázového systému a typu testu.

```
./run_test.sh -d clickhousedb -t insert
```

Ukázka kódu 3.8: Příklad spuštění live insert testu

- **Live insert test**

Test zaměřený na měření rychlosti zápisu dat v reálném čase, kdy se do databáze kontinuálně vkládají nové záznamy. Má své speciální parametry:

- **workers**

Počet paralelních zdrojů, které vkládají data do databáze.

- **duration**

Celková doba v sekundách, po kterou bude test probíhat.

- **batch_size**

Počet záznamů v jedné dávce, která bude vložena do databáze najednou.

Test se zavolá následujícím způsobem:

```
./run_live_insert.sh -d clickhousedb -w 4 -t 300 -b 1000
```

Ukázka kódu 3.9: Příklad spuštění live insert testu

Příkaz nastaví test pro databázový systém ClickHouse přes parametr `-d`, počet zdrojů na 4 přes parametr `-w`, dobu trvání testu na 300 sekund (5 minut) přes parametr `-t` a velikost dávky na 1000 záznamů přes parametr `-b`.

Testování probíhalo na dvou prostředích: lokálně na vývojovém notebooku a vzdáleně na serveru v datovém centru poskytovatele DCUK, kam je přístup realizován přes šifrované VPN²³ připojení. Pro představu o výpočetním kontextu uvádím srovnání klíčových hardwarových parametrů obou strojů v tabulce 3.2.

Tabulka 3.2.: Porovnání hardwarových specifikací testovacích strojů

Parametr	Notebook	Server
Procesor	AMD Ryzen 5 5600H	Intel Xeon E5-2630 v3
Jádra / vlákna	6 / 12	8 / 16
Max. frekvence	4,2 GHz	2,40 GHz
Mezipaměť L3	16 MB	20 MB
Operační paměť	16 GB DDR4-3200	125,69 GB
Úložiště	512 GB NVMe M.2 SSD	1,92 TB Micron SATA SSD
OS	Ubuntu 22.04 LTS (server)	Proxmox VE 9.0.11
Jádro	–	Linux 6.14.11-4-pve

Architektura spouštění testů

Spuštění libovolného testu je řízeno jediným playbookem `tests/test.yml`, který se skládá ze dvou částí. První část zajistí, že cílová databáze běží (role `common-start`, která se v případě neexistujícího virtuálního stroje sama pokusí o jeho vytvoření rolí `common-create`). Druhá část se připojí přímo na databázový host, vyčká na dostupnost databáze a poté vloží konkrétní testovací úlohu podle proměnných `database` a `test_type`:

```
tasks:
  - name: Start {{ database }} - {{ test_type }}
    ansible.builtin.include_tasks:
      file: "{{ _database_ }}/{{ _test_type_ }}.yaml"

post_tasks:
  - name: Common Post Processing
    ansible.builtin.include_role:
      name: common-post-processing
```

Ukázka kódu 3.10: Výřez z `tests/test.yml`

²³VPN (Virtual Private Network) - technologie, která umožňuje bezpečné připojení ke vzdálené síti

Díky tomuto vzoru obsahuje adresář `tests/` pro každou databázi pět souborů se stejným názvem jako typ testu (`insert.yml`, `single_point.yml`, `condition.yml`, `aggregate.yml`, `live_insert.yml`), které obsahují databázově specifický dotaz, ale vracejí shodnou sadu faktů (`start_datetime`, `end_datetime`, `rows_returned...`), se kterou dále pracuje role `common-post-processing`.

Měření doby trvání dotazu

Pro testy *single point*, *condition* a *aggregate* nebyla doba trvání měřena na straně Ansible (tj. jako čas mezi odesláním a přijetím SSH příkazu), ale přímo na straně databázového serveru. U ClickHouse je každý dotaz spuštěn s explicitním `-query_id`, jehož hodnota je následně vyhledána v systémové tabulce `system.query_log`:

```
SELECT
    query_duration_ms ,
    read_rows ,
    result_rows
FROM system.query_log
WHERE type = 'QueryFinish'
    AND query_id = '{{_clickhouse_query_id_}}'
LIMIT 1
```

Ukázka kódu 3.11: Výřez z `tests/clickhousedb/helper/get_query_timing.yml`

Tento přístup, opakovaný s krátkým prodlením do maximálního počtu pokusů (dotaz se v `query_log` objeví až po jeho dokončení a krátkém interním zpoždění), odstraňuje z naměřené hodnoty režii SSH spojení a síťovou latenci mezi řídicím strojem a testovaným kontejnerem, naměřený čas tak odpovídá čistě době zpracování dotazu databázovým enginem.

Test *insert* je naopak měřen na straně shellu pomocí nástroje `/usr/bin/time`, protože importovaná data se do `system.query_log` zapisují jako jediný (dlouho běžící) dotaz a jeho reálná doba trvání zahrnuje i čtení a dekompresi vstupního souboru:

```
/usr/bin/time -f "DURATION_SECONDS=%e" \
sudo clickhouse-client \
    -q "INSERT INTO {{_db_table_}} FORMAT CSVWithNames" \
    < {{ import_dir }}/{{ data_gz }}
```

Ukázka kódu 3.12: Výřez z `tests/clickhousedb/insert.yml`

Live insert test

Test v reálném čase je jediný, který není implementován jako jednorázový SQL příkaz, ale jako samostatný Python skript nasazený na cílový virtuální stroj. Sdílené soubory `generate_live_data.py`, `runner.py` a `base_driver.py` jsou společné pro všechny databáze a definují abstraktní rozhraní

ovladače (počet workerů, batch size a interval měření). Konkrétní implementace zápisu pro danou databázi (např. přes knihovnu `clickhouse-driver`) je uložena v `tests/{database}/driver.py` a je na virtuální stroj zkopírována až podle proměnné `database`:

```
- name: Deploy {{ database }} driver to VM
  ansible.builtin.copy:
    src: "{{_project_root_}}/tests/{{_database_}}/driver.py"
    dest: "{{_bench_dir_path_}}/driver.py"

- name: Run live insert benchmark
  ansible.builtin.command: >
    python3 {{ bench_dir.path }}/runner.py
    --driver-path {{ bench_dir.path }}/driver.py
    --duration {{ duration }} --workers {{ workers }}
    --batch-size {{ batch_size }} --ts-step-ms {{ ts_step_ms }}
    --output {{ bench_dir.path }}/live_result.json
```

Ukázka kódu 3.13: Výřez z `tests/clickhousedb/live_insert.yml`

Skript `runner.py` rozdělí zadaný počet simulovaných vozidel rovnoměrně mezi `workers` paralelních procesů, po dobu `duration` sekund vkládá dávky o velikosti `batch_size` záznamů a výsledek (celkový počet vložených řádků, statistiky za jednotlivé workery) uloží jako JSON. Ansible tento soubor po skončení testu stáhne zpět a jeho hodnoty namapuje na stejnou sadu faktů, kterou používají ostatní testy, díky čemuž prochází live insert test stejnou post-processing logikou jako zbytek testovací sady.

Sběr a agregace výsledků

Bez ohledu na typ testu je posledním krokem každého běhu role `common-post-processing`, která:

- přečte konfiguraci virtuálního stroje uloženou při jeho vytváření v `/etc/vm_config.json` (přidělený RAM, CPU a použitý profil),
- sestaví jeden JSON záznam obsahující typ testu, cílovou databázi, čas začátku a konce, dobu trvání, počet přečtených/vrácených řádků a parametry profilu,
- tento záznam připojí jako nový řádek do souboru `results/{test_type}.ndjson` na řídicím stroji,
- a zapíše odpovídající anotaci do Grafany (`/api/annotations`), čímž se testovací okno zpětně promítne do grafů systémových metrik sbíraných Prometheusem.

Tímto způsobem vznikne za všechny databáze a všechny opakované běhy jednotná sada pěti NDJSON(Newline Delimited JSON) souborů (`insert`, `single_point`, `condition`, `aggregate`, `live_insert`), která slouží jako vstupní data pro následnou analýzu výsledků.

Ze všech testů se sbírají metriky dvojím způsobem:

- **NDJSON výsledky**

Po dokončení každého testu se výsledky ukládají do souboru ve formátu NDJSON, který obsahuje informace o době trvání dotazu, počtu záznamů vrácených dotazem...

- **Prometheus metriky**

Během testů se v každém kontejneru sbírají metriky pomocí Node Exporteru, které jsou následně shromažďovány službou Prometheus. Tyto metriky zahrnují využití CPU, využití paměti, I/O operace disku a síťovou aktivitu.

4. Výsledky výzkumu a diskuse

5. Závěr

Seznam použitých zdrojů

1. DIGGLE, Peter; GIORGI, Emanuele. *Time Series: A Biostatistical Introduction*. 2. vyd. OUP Oxford, 2025. Oxford Statistical Science Series. ISBN 9780191024207. Dostupné také z: <https://books.google.cz/books?id=09s2EQAAQBAJ>.
2. NIELSEN, Aileen. *Practical time series analysis: Prediction with statistics and machine learning*. O'Reilly Media, 2019. ISBN 9781492041658. Dostupné také z: <https://books.google.cz/books?id=odCwDwAAQBAJ>.
3. KLEPPMANN, Martin. *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. O'Reilly Media, 2017. ISBN 9781491903100. Dostupné také z: <https://books.google.cz/books?id=p1heDgAAQBAJ>.
4. CLICKHOUSE. *OLTP vs OLAP: Understanding the Differences* [online]. [cit. 2026-06-29]. Dostupné z: <https://clickhouse.com/resources/engineering/oltp-vs-olap>.
5. MONGODB, Inc. *ACID Transactions in MongoDB* [online]. [cit. 2026-06-29]. Dostupné z: <https://www.mongodb.com/resources/basics/databases/acid-transactions#what-are-acid-transactions>.
6. MICROSOFT. *What is NoSQL database?* [online]. [cit. 2025-11-10]. Dostupné z: <https://azure.microsoft.com/cs-cz/resources/cloud-computing-dictionary/what-is-nosql-database>. Online. Microsoft.
7. MICROSOFT. *What is SQL database?* [online]. [cit. 2025-11-10]. Dostupné z: <https://azure.microsoft.com/cs-cz/resources/cloud-computing-dictionary/what-is-sql-database>. Online. Microsoft.
8. REDIS. *Use Cases | Docs* [online]. [cit. 2025-11-10]. Dostupné z: https://redis.io/docs/latest/develop/data-types/timeseries/use_cases/. Online. Redis.
9. MONGODB. *Use Cases | MongoDB* [online]. [cit. 2025-11-10]. Dostupné z: <https://www.mongodb.com/solutions/use-cases>. Online. MongoDB.
10. CLICKHOUSE. *Use Cases | ClickHouse* [online]. [cit. 2025-11-10]. Dostupné z: <https://clickhouse.com/use-cases>. Online. ClickHouse.
11. NEO4J. *Graph Database Use Cases & Solutions* [online]. [cit. 2025-11-10]. Dostupné z: <https://neo4j.com/use-cases/>. Online. Neo4j.
12. SCYLLADB. *Log-Structured Merge-Tree Definition* [online]. [cit. 2025-11-10]. Dostupné z: <https://www.scylladb.com/glossary/log-structured-merge-tree/>. Online. ScyllaDB.

13. HAT, Red. *Containers vs VMs* [online]. [cit. 2025-11-10]. Dostupné z: <https://www.redhat.com/en/topics/containers/containers-vs-vms>. Online. Red Hat.
14. HAT, Red. *What is a hypervisor?* [online]. [cit. 2025-11-10]. Dostupné z: <https://www.redhat.com/en/topics/virtualization/what-is-a-hypervisor>. Online. Red Hat.
15. INC., Docker. *Docker Docs* [online]. [cit. 2026-06-29]. Dostupné z: <https://docs.docker.com>.
16. CONTAINERS, Linux. *LXD documentation 5.21.5* [online]. [cit. 2025-11-10]. Dostupné z: <https://canonical.com/lxd/docs/default/>. Online. Linux Containers.
17. CONTAINERS, Linux. *Linux Containers - LXC - Introduction* [online]. [cit. 2025-11-10]. Dostupné z: <https://linuxcontainers.org/lxc/introduction/>. Online. Linux Containers.
18. ORACLE. *About Oracle VirtualBox* [online]. [cit. 2026-06-29]. Dostupné z: https://www.virtualbox.org/manual/topics/Introduction.html#ct_about-virtualbox.
19. KRÜGER, Nico. *Agent vs. Agentless: What is better for Infrastructure Management?* [online]. [cit. 2025-11-10]. Dostupné z: <https://www.puppet.com/blog/agent-vs-agentless-security>. Online. Puppet.
20. HOCHSTEIN, Lorin; MOSER, Rene. *Ansible: Up and Running: Automating Configuration Management and Deployment the Easy Way*. O'Reilly Media, 2017. ISBN 9781491979778. Dostupné také z: <https://books.google.cz/books?id=h5YtDwAAQBAJ>.
21. PUPPET. *Puppet Documentation* [online]. [cit. 2025-11-10]. Dostupné z: https://help.puppet.com/core/current/Content/PuppetCore/puppet_index.htm. Online. Puppet.
22. INFLUXDATA. *Telegraf documentation* [online]. [cit. 2025-11-10]. Dostupné z: <https://docs.influxdata.com/telegraf/v1/>. Online. InfluxData.
23. PROMETHEUS. *Prometheus Documentation* [online]. [cit. 2025-11-10]. Dostupné z: <https://prometheus.io/docs/>. Online. Prometheus.
24. MONKEY, Chaos. *Chaos Monkey* [online]. [cit. 2025-11-10]. Dostupné z: <https://netflix.github.io/chaosmonkey/>. Online. Chaos Monkey.
25. YAHOO! *Core Workloads* [online]. [cit. 2026-06-29]. Dostupné z: <https://github.com/brianfrankcooper/YCSB/wiki/Core-Workloads>.
26. COOPER, Brian F.; SILBERSTEIN, Adam; TAM, Erwin; RAMAKRISHNAN, Raghu; SEARS, Russell. Benchmarking Cloud Serving Systems with YCSB. *Proceedings of the 1st ACM Symposium on Cloud Computing*. 2010. Dostupné z DOI: 10.1145/1807128.1807152.
27. COUNCIL, Transaction Processing Performance. *TPC Benchmark C Standard Specification* [online]. 2010. [cit. 2026-06-29]. Tech. zpr. Transaction Processing Performance Council. Dostupné z: https://www.tpc.org/TPC_Documents_Current_Versions/pdf/tpc-c_v5.11.0.pdf. Revision 5.11.

28. JAIN, Raj. *The Art of Computer Systems Performance Analysis: Techniques for Experimental Design, Measurement, Simulation, and Modeling*. Wiley, 1991. ISBN 978-0471503361.
29. CLICKHOUSE. *What is ClickHouse?* [online]. [cit. 2025-11-10]. Dostupné z: <https://clickhouse.com/docs/intro>. Online. ClickHouse.
30. CLICKHOUSE. *Architecture Overview | ClickHouse* [online]. [cit. 2025-11-10]. Dostupné z: https://clickhouse.com/docs/academic_overview. Online. ClickHouse.
31. INFLUXDATA. *InfluxDB Documentation* [online]. [cit. 2025-11-10]. Dostupné z: <https://docs.influxdata.com/influxdb/latest/>. Online. InfluxData.
32. GROUP, PostgreSQL Global Development. *Write-Ahead Logging* [online]. [cit. 2025-11-10]. Dostupné z: <https://www.postgresql.org/docs/current/wal-intro.html>. Online. PostgreSQL.
33. FOUNDATION, Apache Software. *Apache Parquet Documentation* [online]. [cit. 2025-11-10]. Dostupné z: <https://parquet.apache.org/docs/>. Online. Apache Parquet.
34. IOTDB, Apache. *Apache IoTDB Documentation* [online]. [cit. 2025-11-10]. Dostupné z: https://iotdb.apache.org/UserGuide/latest/IoTDB-Introduction/IoTDB-Introduction_apache.html. Online. Apache IoTDB.
35. MARIADB. *MariaDB Documentation* [online]. [cit. 2025-11-10]. Dostupné z: <https://mariadb.com/docs/>. Online. MariaDB.
36. MARIADB. *MariaDB ColumnStore* [online]. [cit. 2025-11-10]. Dostupné z: <https://mariadb.com/docs/analytics/mariadb-columnstore>. Online. MariaDB.
37. MONGODB. *MongoDB Documentation* [online]. [cit. 2025-11-10]. Dostupné z: <https://www.mongodb.com/docs/>. Online. MongoDB.
38. MONGODB, Inc. *WiredTiger Storage Engine* [online]. [cit. 2026-06-29]. Dostupné z: <https://source.wiredtiger.com/11.3.1/overview.html>.
39. QUESTDB. *QuestDB Documentation* [online]. [cit. 2025-11-10]. Dostupné z: <https://questdb.io/docs/>. Online. QuestDB.
40. TIMESCALEDB. *TimescaleDB Documentation* [online]. [cit. 2025-11-10]. Dostupné z: <https://docs.timescale.com/>. Online. TimescaleDB.
41. YAML. *About YAML* [online]. [cit. 2025-11-10]. Dostupné z: <https://yaml.org/about/>. Online. YAML.
42. JINJA2. *Jinja2 Documentation* [online]. [cit. 2025-11-10]. Dostupné z: <https://jinja.palletsprojects.com/>. Online. Jinja2.
43. HAT, Red. *Ansible Community | Ansible documentation* [online]. [cit. 2025-11-10]. Dostupné z: <https://docs.ansible.com>. Online. Red Hat.

A. Architektura Ansible

V Ansible jsou pro definici konfigurace a automatizačních úloh nejčastěji používány strukturované formáty YAML, INI a šablonovací jazyk Jinja2.

- **YAML**

YAML (YAML Ain't Markup Language) je formát pro serializaci dat, který je navržen tak, aby byl snadno čitelný pro lidi a zároveň snadno zpracovatelný počítači. [41]

```
general:
  name: TestApp
  version: 1.0

settings:
  theme: dark
  lang: cs
```

Ukázka kódu A.1: Ukázka YAML

- **INI**

INI je jednoduchý formát pro konfigurační soubory, který používá strukturu klíč-hodnota a sekce pro organizaci nastavení.

```
[General]
name=TestApp
version=1.0

[Settings]
theme=dark
lang=cs
```

Ukázka kódu A.2: Ukázka INI

- **Jinja2**

Jinja2 je šablonovací jazyk pro Python, který umožňuje generovat textové soubory na základě šablon a proměnných. [42]

```
Application: {{ name }}
Version: {{ version }}

Language: {{ lang }}
Theme: {{ theme }}
```

Ukázka kódu A.3: Ukázka Jinja2

Tyto formáty tvoří základ pro hlavní části konfigurace, jako jsou playbooky, inventáře a šablony. Kromě nich však mohou být v rámci Ansible projektů využívány také další typy souborů, například skripty v jazycích Bash nebo Python, či statické datové soubory (např. JSON nebo CSV), které slouží jako doplňkové zdroje dat. [43]

Samotná struktura Ansible projektu je následně rozdělena do několika klíčových komponent:

- **Řídící uzel**

Řídící uzel je stroj, na kterém je nainstalován Ansible a odkud jsou spouštěny všechny automatizační úlohy. Obsahuje veškeré konfigurační soubory, playbooky a inventáře cílových uzlů. [43]

- **Řízené uzly**

Řízené uzly jsou servery nebo zařízení, která Ansible spravuje. Mohou to být například fyzické servery, virtuální stroje, kontejnery nebo síťová zařízení.

Ansible komunikuje s těmito uzly hlavně pomocí protokolu SSH a provádí na nich definované úlohy. Existují ale i jiné způsoby komunikace, například `local` pro lokální operace nebo `docker` pro správu Docker kontejnerů. [43]

V Ansible se dá připojit až už k jednomu nebo více řízeným uzlům najednou. Pokud není specifikováno jinak, tak paralelně. Tyto skupiny jsou definovány v inventáři.

- **Task**

Task, neboli úloha, je základní jednotka práce v Ansible, která definuje konkrétní akci, kterou má Ansible provést na řízeném uzlu. Úlohy jsou definovány v playboocích a mohou zahrnovat různé operace, jako je instalace softwaru, kopírování souborů, spouštění příkazů nebo konfigurace služeb. [43]

```
- name: Ensure postgresql is at the latest version
  ansible.builtin.yum:
    name: postgresql
    state: latest
```

Ukázka kódu A.4: Ansible - ukázka úlohy (Task)

• Playbook

Playbooky jsou soubory ve formátu YAML, které obsahují sadu úloh pro Ansible. Definují, jaké úlohy mají být provedeny na kterých řízených uzlech a v jakém pořadí. Playbooky umožňují automatizovat složité procesy a zajišťují opakovatelnost a konzistenci konfigurací. [43]

```
---
- name: Update db servers
  hosts: dbservers
  remote_user: root

  tasks:
    - name: Ensure postgresql is at the latest version
      ansible.builtin.yum:
        name: postgresql
        state: latest

    - name: Ensure that postgresql is started
      ansible.builtin.service:
        name: postgresql
        state: started
```

Ukázka kódu A.5: Ansible - ukázka playbooku

• Modul

Moduly představují základní vykonávací jednotky v Ansible. Každá úloha (*task*) v playbooku využívá některý modul k provedení konkrétní operace na řízeném uzlu. Modul tedy definuje, jaký typ akce má být vykonán.

Ansible obsahuje velké množství vestavěných modulů, které jsou dostupné v tzv. *kolekcích*. Nejčastěji používané vestavěné moduly se nacházejí v kolekci `ansible.builtin`, například:

- `ansible.builtin.copy` – kopírování souborů mezi řídicím a řízeným uzlem,
- `ansible.builtin.user` – správa uživatelských účtů na řízeném uzlu,
- `ansible.builtin.service` – správa systémových služeb (start, stop, restart),

Lze najít a stáhnout i moduly z externích modulovacích kolekcí. [43]

• Inventář

Inventář je soubor, který obsahuje seznam řízených uzlů, které Ansible spravuje. Může být ve formátu INI, nebo YAML a buď statický, předem sestavený, nebo dynamický, vygenerovaný skriptem. Inventář umožňuje organizovat uzly do skupin, což usnadňuje cílení úloh na specifické sady zařízení. [43]

```
mail.example.com

[webservers]
foo.example.com
bar.example.com

[dbservers]
one.example.com
two.example.com
192.168.1.200
```

Ukázka kódu A.6: Ansible - ukázka INI inventáře

```
ungrouped:
  hosts:
    mail.example.com:
webservers:
  hosts:
    foo.example.com:
    bar.example.com:
dbservers:
  hosts:
    one.example.com:
    two.example.com:
    192.168.1.200:
```

Ukázka kódu A.7: Ansible - ukázka YAML inventáře

Ansible definuje vždy 2 hlavní seskupení v inventáři:

- **all** - obsahuje všechny zařízení v inventáři
- **ungrouped** - obsahuje zařízení, které nejsou přiřazeny do žádné skupiny

Všechny ostatní skupiny jsou definovány uživatelem a mohou být hierarchicky organizovány, což umožňuje vytvářet podskupiny a dědit vlastnosti mezi skupinami. [43]

- **Role**

Role jsou způsob, jak organizovat playbooky a úlohy do znovupoužitelných komponent. Takto vytvořené role mohou být dále znovu použity a také sdíleny přes Ansible Galaxy. Role umožňují strukturovat kód do předem definovaných adresářů, které vypadají následovně:

```
roles/  
  common/  
    tasks/  
      main.yml  
    handlers/  
      main.yml  
    templates/  
      ntp.conf.j2  
    files/  
      bar.txt  
      foo.sh  
    vars/  
      main.yml  
    defaults/  
      main.yml  
    meta/  
      main.yml  
    library/  
    module_utils/  
    lookup_plugins/  
  
  webtier/  
  monitoring/  
  fooapp/
```

Ukázka kódu A.8: Struktura adresářů pro Ansible role

Jednotlivé adresáře v roli mají specifický účel:

- **tasks** - obsahuje hlavní úlohy, které se mají provést, obvykle v souboru `main.yml`
- **handlers** - obsahuje úlohy, které se spustí pouze v případě, že jsou "vyvolány" z jiných úloh, například pro restart služby po změně konfigurace
- **templates** - obsahuje šablony, které se používají pro generování konfiguračních souborů na základě proměnných
- **files** - obsahuje statické soubory, které se mají zkopírovat na řízené uzly
- **vars** - obsahuje proměnné, které se mají použít v souboru `main.yml`
- **defaults** - obsahuje proměnné, které se mají použít v souboru `main.yml`, ale s nižší prioritou než proměnné v adresáři `vars`
- **meta** - obsahuje metadata o roli, jako jsou závislosti na jiných rolích
- **library** - obsahuje vlastní moduly, které mohou být použity v úlohách
- **module_utils** - obsahuje pomocné funkce pro vlastní moduly
- **lookup_plugins** - obsahuje vlastní lookup pluginy, které umožňují získávat data z různých zdrojů

• Proměnná

Proměnné v Ansible umožňují ukládat hodnoty, které lze použít v různých částech playbooku. Tyto proměnné mohou být definovány na různých místech v projektu, a podle toho mají i přiřazené priority, pro případ kdyby se proměnné jmenovaly stejně (např. při definování výchozí cesty k souboru s konfigurací). [43]

Název proměnných je pouze omezen tím, že musí začínat písmenem nebo podtržítkem a může obsahovat pouze alfanumerické znaky a podtržítka. Dále jsou některé názvy proměnných rezervované, například `ansible_facts` nebo `inventory_hostname`. [43]

Proměnné se používají pomocí šablonovacího systému Jinja2, který umožňuje vkládat proměnné do textu a také provádět různé operace s nimi, jako jsou podmínky, smyčky nebo filtry. Aby Ansible věděl, že se jedná o proměnnou, musí být její název obklopen dvojími složenými závorkami a uzavřen v uvozovkách, například `"{{ promenna }}"`. [43]

```
- debug :  
  msg: "Hodnota je {{ promenna }}"
```

Ukázka kódu A.9: Použití proměnné

– Fakta

Specifickým typem proměnných jsou tzv. "fakta", které Ansible automaticky shromažďuje o cílových uzlech během běhu playbooku.

Přístupuje se k nim přes proměnnou `ansible_facts` a obsahují informace o hardwaru, operačním systému, síťových rozhraních a dalších charakteristikách uzlu. K některým

faktům lze přistupovat i zkráceně, například `ansible_facts['os_family']` lze zapsat jako `ansible_os_family`. [43]

```
{
  "ansible_all_ipv4_addresses": [
    "192.168.1.2"
  ],
  "ansible_all_ipv6_addresses": [
    "2345:0425:2CA1:0000:0000:0567:5673:23b5"
  ],
  "ansible_apparmor": {
    "status": "disabled"
  },
  "ansible_architecture": "x86_64",
  ...
}
```

Ukázka kódu A.10: Shromážděná fakta

Shromažďování faktů lze ovlivnit pomocí proměnné `gather_facts`, která se nastavuje na úrovni playbooku. Jejich kolekce může pro některé případy být zbytečná a velmi zpomalovat průběh playbooku. [43]

Při kolizi názvů proměnných se použije následující neuplný výčet priority (vyšší číslo znamená vyšší prioritu):

1. Proměnné přidáné argumentem

```
ansible-playbook playbook.yml -e "promenna=foo"
```

Ukázka kódu A.11: Proměnné přidáné argumentem

2. Proměnné role

Proměnné definované ve `roles/myrole/vars/main.yml`. Na stejné úrovni jsou i proměnné definované při zahrnutí role v playbooku.

```
- roles:
  - role: myrole
    vars:
      promenna: foo
```

Ukázka kódu A.12: Proměnné při zahrnutí role

3. Registrované proměnné

```
- command: echo hello
register: promenna
```

Ukázka kódu A.13: Registrované proměnné

Tyto proměnné jsou vytvořeny během běhu playbooku a obsahují výsledky z úloh, které je vytvořily. Využívají se například pro kontrolu úspěšného provedení úlohy nebo pro získání výstupu z příkazu.

4. Playbook proměnné

Proměnné definované přímo v souboru `playbook.yml`.

```
- hosts: all
  vars:
    promenna: foo
  tasks:
    ...
```

Ukázka kódu A.14: Playbook proměnné

5. Host_vars proměnné

Proměnné definované pro konkrétní hosty v adresáři `host_vars/`. Každý soubor v tomto adresáři je pojmenován podle názvu hosta, pro který jsou proměnné určeny.

```
# host_vars/server1.yml
promenna: foo
```

Ukázka kódu A.15: Proměnné hostů

6. Group_vars proměnné

Proměnné definované pro skupiny hostů v adresáři `group_vars/`. Každý soubor v tomto adresáři je pojmenován podle názvu skupiny, pro kterou jsou proměnné určeny.

```
# group_vars/web.yml
promenna: foo
```

Ukázka kódu A.16: Proměnné skupin

7. Proměnné v inventáři

Proměnné definované přímo v inventářním souboru. Může se jednat o proměnné pro konkrétní hosty nebo pro celé skupiny podle umístění. Využívají se například pro definování specifických nastavení pro jednotlivé skupiny, jako je např. styl připojení.

```
[web]
server1 promenna=foo
```

Ukázka kódu A.17: Proměnné v inventáři

8. Fakta

Automaticky shromážděné informace o cílových uzlech, přístupné přes proměnnou `ansible_facts`.

9. Výchozí proměnné role

Definované v `roles/myrole/defaults/main.yml`. Absolutně nejnižší priorita.

Citlivé proměnné, jako jsou hesla nebo klíče, by měly být šifrovány pomocí Ansible Vault, což je nástroj pro zabezpečení dat v Ansible.

Ten ale zabezpečuje data jen v případě "data at rest", to znamená uložená data v souborech. Jakmile jsou data během běhu playbooku dešifrována, nacházejí se v paměti a mohou být zobrazeny ve výstupu. Proto je vhodné citlivé informace skrýt pomocí parametru `no_log`, který zabrání jejich zobrazení ve výstupu playbooku. [43]

B. Architektura Prometheus

Prometheus se konfiguruje pomocí YAML souboru, kde se definují zdroje dat, pravidla pro shromažďování metrik a nastavení upozornění.

```
global:
  scrape_interval: 15s
  evaluation_interval: 15s

rule_files:
  - "alert.rules"

scrape_configs:
- job_name: prometheus
  static_configs:
  - targets: ['localhost:9090']
```

Ukázka kódu B.1: Prometheus - ukázka konfigurace

Existují zde 3 hlavní bloky:

- **global** - obsahuje globální nastavení pro Prometheus, jako je interval shromažďování dat a interval vyhodnocování pravidel pro upozornění.
- **rule_files** - obsahuje seznam souborů s pravidly pro upozornění, které definují podmínky, za kterých má být odesláno upozornění.
- **scrape_configs** - obsahuje konfigurace pro shromažďování dat z různých zdrojů, včetně názvu úlohy a seznamu cílů, ze kterých se mají data shromažďovat.

Datový Model

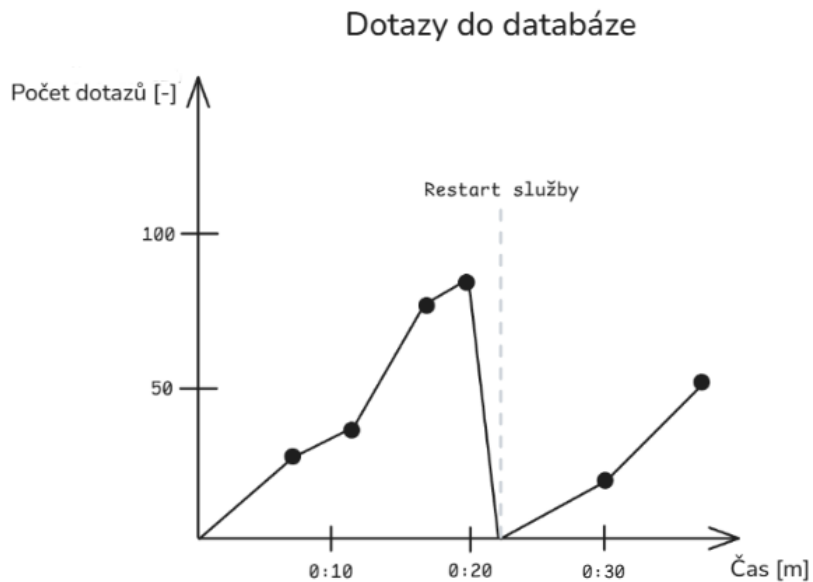
Prometheus ukládá data ve formě časových řad.

- **Metrika** - základní jednotka dat, která představuje konkrétní sadu měření, například `http_requests_total` pro počet HTTP požadavků.
- **Štítek** - klíč-hodnota pár, který poskytuje další kontextové informace o metrice.
- **Hodnota** - konkrétní měření pro danou metriku a sadu štítků, například počet požadavků v určitém časovém okamžiku.

Dále se jednotlivé metriky dělí do 4 typů:

- **Counter**

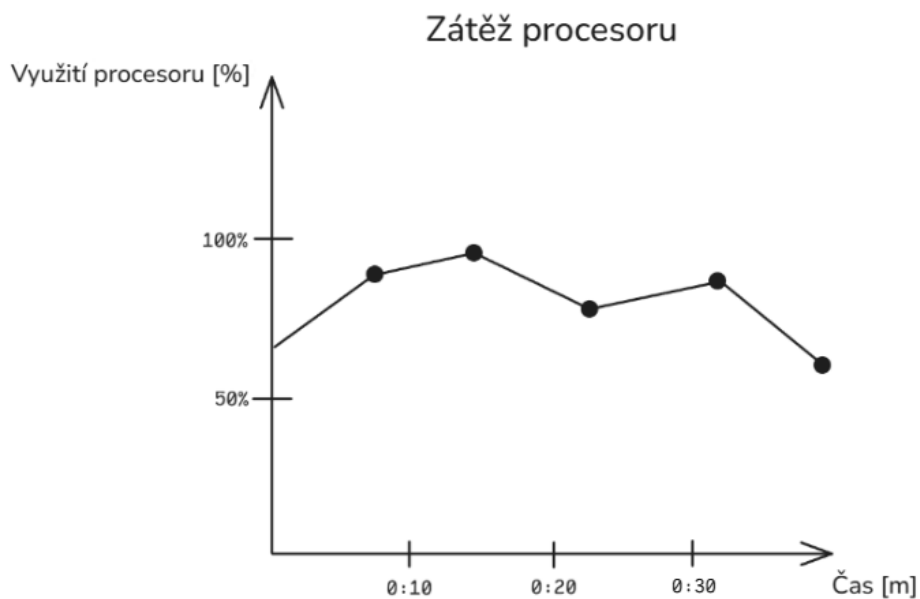
Monotonicky rostoucí hodnota, která se zvyšuje pouze při události, například počet požadavků.



Obrázek B.1.: Ukázka metriky typu Counter (zdroj: vlastní zpracování)

- **Gauge**

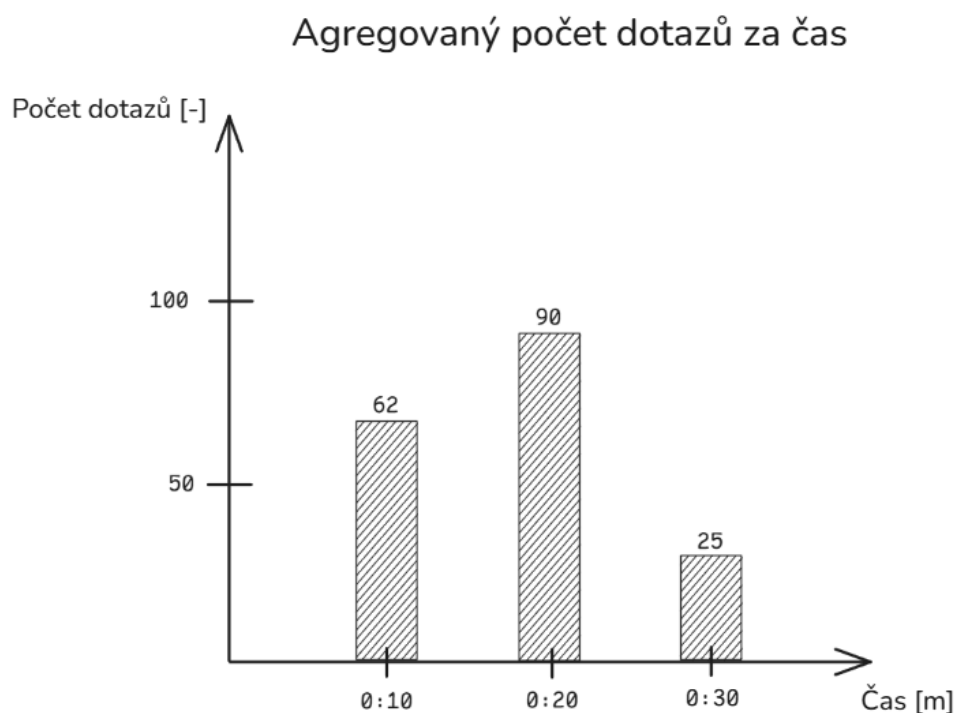
Hodnota, která může růst i klesat, například aktuální využití CPU.



Obrázek B.2.: Ukázka metriky typu Gauge (zdroj: vlastní zpracování)

- **Histogram**

Měří distribuci hodnot, například doby odezvy, a poskytuje informace o počtu vzorků a součtech pro různé intervaly.



Obrázek B.3.: Ukázka metriky typu Histogram (zdroj: vlastní zpracování)

- **Summary**

Podobný histogramu, ale poskytuje kvantily pro měření distribuce, například 95. percentil doby odezvy.

PromQL

PromQL, celým názvem Prometheus Query Language, je dotazovací jazyk pro Prometheus. Dotazy mohou být buď "instantní", které vracejí aktuální hodnoty metrik, nebo "časový rozsah", které vracejí historické hodnoty pro zadané časové období. [23]

```
rate(http_requests_total[5m])[30m:1m]
```

Ukázka kódu B.2: PromQL - ukázka dotazů

Tento dotaz vrací rychlost změny metriky `http_requests_total` za posledních 5 minut, přičemž se tato rychlost počítá pro každou minutu v posledních 30 minutách.

Node Exporter

Node Exporter je nástroj pro sběr systémových metrik z hostitelského systému, který je monitorován pomocí Prometheus. Node Exporter shromažďuje informace o specifickém systému pro který je konfigurován, například o CPU, paměti, disku, síti a dalších hardwarových a systémových parametrech. [23]

Je třeba ho distribuovat a spustit na každém hostitelském systému, který chceme monitorovat, a poté nakonfigurovat Prometheus, aby shromažďoval data z těchto Node Exporterů. [23]

Alertmanager

Alertmanager je komponenta Prometheus, která se stará o správu a odesílání upozornění. Umožňuje definovat pravidla pro upozornění, která se spouštějí na základě dotazů PromQL, a poté odesílat upozornění prostřednictvím různých kanálů, jako jsou e-mail, Slack a další. [23]

C. Externí přílohy

Externí přílohy této bakalářské práce jsou umístěny na adrese:

https://github.com/Jiri-Fiser/thesis_ki_ujep.

Na úložišti GitHub mohou být uloženy tyto externí přílohy:

- **zdrojové kódy**
- **doplňkové texty** (například jak instalovat aplikaci, manuály aplikace)
- **schémata** (především, pokud se nevejdou na stranu A4 a jejich vytištění je tak problematické)
- **screenshoty** (v textu práce lze použít jen omezený počet snímků obrazovky, které navíc nemusí být při černobílém tisku příliš přehledné)
- **videa** (například ovládání aplikace)

V každém případě by to však měli být pouze materiály, které jste vytvořili sami. Materiály jiných autorů uvádějte v seznamu použité literatury (včetně případných odkazů na jejich originální umístění).

V této kapitole stačí uvést pouze základní strukturu úložiště (co se kde nalézá a jakou má funkci) například v podobě tabulky.

ki-thesis.pdf	text práce v PDF
ki-thesis.tex	zdrojový kód práce v $\text{\LaTeX}$
kitheses.cls	definice třídy dokumentů (rozšířená třída scrbook)
thesis.bib	bibliografická databáze (exportována z citace.com)
LOGO_PRF_CZ_RGB_standard.jpg	logo fakulty s českým textem
LOGO_PRF_EM_RGB_standard.jpg	logo fakulty s anglickým textem

Všechny tyto soubory jsou potřeba pro překlad dokumentu (logo stačí jedno v příslušné jazykové verzi).